

АННОТАЦИЯ

ВКР бакалавра «Разработка серверной части веб-приложения для заметок в учебном процессе.».

Пояснительная записка: 85 стр., 16 рис., 1 табл., 31 источн., 1 прилож.

Ключевые слова: веб-приложение, заметки, Python, Django, микросервис, системы дистанционного обучения.

Цель работы – разработка эффективного и безопасного веб-приложения для управления заметками.

Приложение предназначено для студентов и преподавателей учебных заведений, а также для пользователей, нуждающихся в эффективном управлении заметками для образовательных целей.

В качестве основного инструмента для разработки был использован фреймворк Django. API разрабатывалось следуя принципам REST. Для базы данных использовался SQLite, для более быстрой разработки.

Приложение позволяет пользователям проходить аутентификацию, создавать, редактировать и удалять заметки, присваивать им теги и использовать папки для их категоризации. Присутствуют функции поиска и сортировки.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ПОСТАНОВКА ЗАДАЧИ	8
2 ОБЗОР ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ ПО ТЕМАТИКЕ ПРОЕКТИРОВАНИЯ	10
2.1 Исследование процесса разработки серверной части приложения	10
2.2 Преимущества и недостатки Python	12
2.3 Django	16
2.4 Сравнение СУБД	20
2.5 Изучение SaaS модели	23
3 СИСТЕМНЫЙ АНАЛИЗ СЕРВЕРНОЙ ЧАСТИ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ЗАМЕТОК	26
3.1 Определение объекта и предмета анализа	26
3.2 Функции и структура исследуемого объекта	27
3.3 Выделение подсистем в объекте исследования	32
3.4 Определение связей между подсистемами исследуемого объекта и внешней средой	39
3.5 Системотехнические принципы	42
4 РАЗРАБОТКА СИСТЕМЫ	47
4.1 Выбор технологий разработки	47
4.2 Основные этапы разработки	49
5 ТЕСТИРОВАНИЕ СИСТЕМЫ	58
5.1 Тестирование аутентификации	58
5.2 Тестирование заметок	60
5.3 Тестирование тегов	62
5.4 Тестирование папок	64
ЗАКЛЮЧЕНИЕ	67

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ	68
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	69
ПРИЛОЖЕНИЕ А	72

ВВЕДЕНИЕ

Данная выпускная квалификационная работа посвящена созданию серверной части веб-приложения для заметок, ориентированного на образовательные цели.

Современный образовательный процесс все глубже погружается в цифровую среду и в данный момент требует эффективных инструментов для организации и управления информацией, таких как заметки. Это приводит к тому, что веб-приложения для управления заметками становятся неотъемлемой частью образовательного процесса, предоставляя студентам и преподавателям удобные средства для хранения, структурирования и совместной работы с данными.

Разработка серверной части таких приложений является сложной задачей, требующей тщательного анализа требований, выбора подходящих технологий и обеспечения безопасности, производительности и масштабируемости.

Разработка основана на использовании фреймворка Django, который обеспечивает быструю разработку, высокую безопасность и поддержку модульной архитектуры. В работе рассматриваются этапы системного анализа, проектирования, реализации и тестирования системы, а также обосновывается выбор технологий, таких как Python, Django и SQLite (для разработки), с возможностью перехода на PostgreSQL. Часть работы посвящена архитектуре REST API, обеспечивающей взаимодействие с клиентской частью, и подсистемам, отвечающим за аутентификацию, управление заметками, папками и тегами.

1 ПОСТАНОВКА ЗАДАЧИ

Разработать архитектурную схему серверной части приложения. Определить структуру базы данных, включая таблицы для пользователей, заметок, категорий и тегов.

Спроектировать API, используя RESTful подход, для взаимодействия с клиентскими приложениями

Настроить проект Django и создать необходимые приложения. Реализовать модели данных для базы данных. Разработать маршруты для обработки запросов к API.

Разработать серверную часть приложения заметок, которая:

- Поддерживает базовые операции с заметками (создание, редактирование, удаление, поиск).
- Обеспечивает иерархическую организацию заметок через папки и категоризацию через теги.
- Гарантирует безопасность данных (шифрование, аутентификация, авторизация).
- Обеспечивает производительность и масштабируемость.

Требования к системе:

Функциональные:

- Регистрация и аутентификация пользователей.
- CRUD-операции для заметок, папок, тегов.
- Иерархическая структура папок.
- Фильтрация заметок по тегам, папкам, датам.
- API для взаимодействия с клиентской частью.

Нефункциональные:

- Безопасность: HTTPS/TLS, защита от SQL-инъекций, XSS, CSRF.
- Надежность: Резервное копирование, отказоустойчивость

Ограничения:

- Технологические: Использование Django и SQLite(для разработки).

- Ресурсные: Ограниченный объем вычислительных ресурсов для прототипа (локальное развертывание).
- Образовательный контекст: Фокус на сценариях совместной работы студентов и преподавателей.

2 ОБЗОР ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ ПО ТЕМАТИКЕ ПРОЕКТИРОВАНИЯ

2.1 Исследование процесса разработки серверной части приложения

Серверная часть приложения — это компонент программного обеспечения, который обрабатывает запросы от клиентов и выполняет бизнес-логику, взаимодействуя с базами данных и другими сервисами. Она отвечает за выполнение операций, которые не видны пользователю, но необходимы для функционирования приложения [1].

Основные функции серверной части:

1. **Обработка запросов:** сервер принимает запросы от клиентов и обрабатывает их. Это может включать в себя получение данных, выполнение операций и отправку ответов.
2. **Бизнес-логика:** серверная часть реализует бизнес-логику приложения, то есть правила и процессы, которые определяют, как данные обрабатываются и какие действия выполняются в ответ на запросы.
3. **Взаимодействие с базами данных:** серверная часть часто взаимодействует с базами данных для хранения, извлечения и обновления данных. Это может включать выполнение SQL-запросов или использование ORM для работы с данными.
4. **Аутентификация и авторизация:** серверная часть отвечает за проверку подлинности пользователей и управление доступом к различным ресурсам приложения.
5. **Обработка ошибок:** серверная часть должна обрабатывать ошибки и исключения, чтобы обеспечить стабильную работу приложения и предоставить пользователям понятные сообщения об ошибках.
6. **Интеграция с внешними сервисами:** сервер может взаимодействовать с другими сервисами и API для получения данных или

выполнения операций, таких как отправка уведомлений, обработка платежей и т.д.

Процесс разработки серверной части приложения:

1. Анализ требований: на этом этапе разработчики работают с клиентами и заинтересованными сторонами, чтобы определить требования к приложению и его функциональности, формируют техническое задание. В результате, по окончании данного этапа должны быть определены: список функций; документ с требованиями; предварительный стек технологий [2, 14].

2. Проектирование архитектуры: на этом этапе происходит выбор архитектурного подхода, определяются используемые технологии, начинается проектирование API и базы данных. По завершению данного этапа должны быть определены: архитектурная схема; спецификация API; структура базы данных.

3. Разработка серверной части: на этом этапе производится написание кода, реализующего бизнес-логику, взаимодействие с базой данных и API. По окончании данного этапа получается рабочий серверный код, в котором реализованы требуемые функции.

4. Тестирование: на этом этапе проводится тестирование приложения для проверки работоспособности и производительности приложения. По завершению данного этапа выявляются и исправляются различные баги.

5. Развертывание: на этом этапе выполняется запуск серверной части приложения для предоставления доступа пользователям. По окончании данного этапа приложение должно быть доступно в сети Интернет.

6. Обслуживание и развитие: на этом этапе приложение уже используют пользователи и происходит поддержание серверов и приложения в работоспособном состоянии, производится масштабирование. Также команда разработки создает обновления и расширяет функционал приложения.

Серверная часть веб-приложения является ключевым компонентом, обеспечивающим обработку запросов, реализацию бизнес-логики, взаимодействие с базами данных, аутентификацию, интеграцию с внешними

сервисами и обработку ошибок. Она скрыта от пользователей, но играет критическую роль в стабильной и эффективной работе приложения. Каждый этап разработки направлен на создание надежного, масштабируемого и функционального продукта, который отвечает потребностям пользователей и бизнеса.

2.2 Преимущества и недостатки Python

Python представляет собой высокоуровневый язык программирования, отличающийся простотой изучения и значительной мощностью. Он предоставляет эффективные структуры данных, поддерживает объектно-ориентированное программирование с использованием лаконичного, но функционального подхода. Благодаря элегантному синтаксису, динамической типизации и интерпретируемой природе Python является оптимальным выбором для создания сценариев и быстрой разработки приложений в различных областях и на большинстве платформ.

Преимущества Python:

- Синтаксис Python характеризуется высокой читаемостью, что делает код, написанный на этом языке, схожим с текстом на английском языке, но при этом строгим и структурированным. Эта особенность позволяет разработчикам сосредотачиваться на решении задач, а не на особенностях языка. Простота синтаксиса делает Python исключительно доступным для начинающих программистов [3].

- Python является свободным программным обеспечением с открытым исходным кодом. Это означает, что пользователи имеют право свободно распространять, изучать, модифицировать и использовать исходный код в своих проектах. Данная модель основана на принципах сообщества, которое активно развивает и совершенствует язык, что существенно способствует его популярности и качеству.

– Python освобождает разработчиков от необходимости управления низкоуровневыми аспектами, такими как выделение памяти, что упрощает процесс программирования. Благодаря своей универсальности и портируемости Python адаптирован для работы на множестве платформ. Программы, написанные на Python, могут исполняться без изменений на различных системах, при условии избегания системно-зависимых функций.

– В отличие от компилируемых языков, таких как C или C++, Python не требует преобразования исходного кода в бинарный формат. Вместо этого Python интерпретирует исходный текст, преобразуя его в промежуточный байткод, который затем исполняется. Это устраняет необходимость компиляции, упрощает процесс разработки и повышает переносимость программ, позволяя запускать их на любом совместимом устройстве путем простой передачи исходного кода [4].

– Python поддерживает как процедурное, так и объектно-ориентированное программирование. Процедурный подход основан на использовании функций, представляющих многократно используемые фрагменты кода, тогда как объектно-ориентированный подход опирается на объекты, объединяющие данные и функциональность. По сравнению с такими языками, как C++ или Java, Python предлагает более простые и эффективные средства для реализации объектно-ориентированного программирования [7].

– Для задач, требующих высокой производительности или сокрытия алгоритмов, Python позволяет интегрировать код, написанный на C или C++, что обеспечивает гибкость в разработке. Кроме того, Python может быть встроен в приложения на C/C++ для предоставления функциональности скриптового программирования.

– Стандартная библиотека Python отличается обширностью и универсальностью, охватывая широкий спектр задач, включая работу с регулярными выражениями, генерацию документации, модульное тестирование, параллельные вычисления, базы данных, веб-браузеры, CGI,

FTP, электронную почту, XML, HTML, WAV-файлы, криптографию, графические интерфейсы и системно-зависимые операции.

Python – мощный и универсальный язык программирования, но, как и любая технология, он имеет свои недостатки:

- Python является интерпретируемым языком, что означает, что код выполняется построчно без предварительной компиляции в машинный код. Это делает Python значительно медленнее по сравнению с компилируемыми языками, такими как C или C++, следовательно Python не подходит для задач, где критически важна скорость выполнения.

- В стандартной реализации Python используется Global Interpreter Lock (GIL), который предотвращает одновременное выполнение нескольких потоков в одном процессе. Это ограничивает возможности многопоточного программирования, что делает Python менее эффективным для задач, требующих интенсивного использования многопоточности.

- Python использует динамическую типизацию и автоматическое управление памятью, что приводит к высокому потреблению оперативной памяти по сравнению с языками, такими как C или C++, следовательно Python не оптимален для приложений, где критически важна экономия памяти.

- Python плохо подходит для разработки приложений под мобильные платформы и встроенные системы из-за отсутствия поддержки данных платформ по умолчанию и высокой ресурсоемкости, следовательно Python не является предпочтительным выбором для мобильных и встроенных приложений.

- Экосистема Python включает множество библиотек, доступных через менеджер пакетов `pip`, но управление зависимостями может быть проблематичным из-за конфликтов версий и отсутствия строгой изоляции окружений, следовательно управление зависимостями в Python требует дополнительных усилий, особенно в крупных проектах.

- Динамическая типизация Python упрощает разработку, но может привести к ошибкам, которые обнаруживаются только во время выполнения,

следовательно динамическая типизация снижает надежность кода в больших проектах.

- Приложения на Python, особенно с использованием множества библиотек, могут иметь большой размер из-за включения зависимостей и интерпретатора, следовательно Python неэффективен для приложений, где важен минимальный размер дистрибутива.

- Простота и гибкость Python могут привести к менее строгому подходу к написанию кода, особенно в командах с низкой дисциплиной, следовательно Python требует дисциплины для поддержания качества кода в больших проектах.

- Несмотря на универсальность Python, в некоторых специализированных областях (например, системное программирование, разработка драйверов, высокопроизводительные игры) он уступает другим языкам, следовательно Python не подходит для узкоспециализированных задач, требующих прямого доступа к оборудованию.

Python – высокоуровневый, универсальный язык программирования, отличающийся простотой, читаемым синтаксисом и поддержкой как процедурного, так и объектно-ориентированного программирования. Его интерпретируемая природа, динамическая типизация и обширная стандартная библиотека делают его идеальным для быстрой разработки приложений в различных областях, от веб-разработки до анализа данных. Python обеспечивает портируемость, освобождает разработчиков от низкоуровневого управления памятью и поддерживается активным сообществом, что способствует его популярности и развитию как свободного ПО с открытым исходным кодом. Возможность интеграции с C/C++ и встраивания в другие приложения добавляет гибкости.

Однако Python имеет существенные недостатки. Его интерпретируемая природа и GIL ограничивают производительность и многопоточность, делая язык менее подходящим для высокопроизводительных задач. Высокое потребление памяти, слабая поддержка мобильных и встроенных систем,

сложности с управлением зависимостями и динамическая типизация, приводящая к ошибкам на этапе выполнения, усложняют использование в крупных или ресурсоемких проектах. Кроме того, Python неэффективен для системного программирования и приложений с жесткими требованиями к размеру дистрибутива, а его гибкость требует строгой дисциплины для поддержания качества кода. Таким образом, Python оптимален для задач, где простота и скорость разработки важнее производительности, но требует тщательного учета его ограничений в специализированных или высоконагруженных сценариях.

2.3 Django

Django – это свободный и открытый фреймворк (open-source), который следует архитектурному паттерну Model-Template-View, являющемуся вариацией Model-View-Controller. Он разработан для упрощения создания сложных, управляемых базами данных веб-приложений, минимизируя необходимость повторного написания стандартных функций. Основная цель Django – ускорить разработку, обеспечивая при этом чистый, прагматичный дизайн и высокий уровень безопасности. Фреймворк поддерживается некоммерческой организацией Django Software Foundation, которая координирует его развитие и организует сообщество разработчиков.

Django используется такими крупными платформами, как Instagram, Pinterest, Mozilla, Disqus и Spotify, что подтверждает его надежность и масштабируемость для проектов любого размера – от небольших сайтов до высоконагруженных приложений.

2.3.1 Ключевые особенности Django

Философия «всё включено»: Django поставляется с множеством встроенных компонентов для решения типичных задач веб-разработки: аутентификация, управление пользователями, администрирование контента,

генерация RSS-лент, карты сайта и многое другое. Это избавляет разработчиков от необходимости подключать сторонние библиотеки для базовых функций.

Объектно-реляционное отображение: Django предоставляет мощный ORM, который позволяет описывать модели данных в виде Python-классов и взаимодействовать с базой данных без написания SQL-запросов.

Административная панель: Django автоматически генерирует интерфейс администратора, который позволяет управлять данными (без дополнительной разработки). Эта панель настраивается и подходит как для разработчиков, так и для контент-менеджеров.

Шаблонизатор (Django Template Language): Django включает собственный язык шаблонов, который позволяет комбинировать статический HTML с динамическими данными. Шаблоны просты в использовании и безопасны, так как предотвращают инъекции кода.

Безопасность: Django имеет встроенные механизмы защиты от распространенных уязвимостей, таких как SQL-инъекции, межсайтовый скриптинг (XSS) и подделка межсайтовых запросов (CSRF). Например, система аутентификации хранит пароли в хэшированном виде, а сессии защищены от компрометации. Фреймворк оперативно выпускает патчи безопасности, что делает его одним из первых, кто реагирует на уязвимости.

URL-маршрутизация: Django использует регулярные выражения для создания чистых и читаемых URL-адресов без лишних расширений. Это улучшает SEO и удобство пользователей.

Поддержка масштабируемости: Django позволяет создавать приложения, способные обрабатывать высокие нагрузки. Такие платформы, как Instagram, успешно используют Django для обслуживания миллионов пользователей [5].

Поддержка REST API: с помощью Django REST Framework можно быстро создавать мощные и масштабируемые API для взаимодействия с клиентской частью или мобильными приложениями. DRF поддерживает сериализацию, аутентификацию и веб-интерфейс для тестирования API.

Экосистема и расширяемость: существует более 5000 сторонних пакетов, доступных через Django Packages, которые расширяют функциональность фреймворка. Django поддерживает интеграцию с другими технологиями, такими как JavaScript-фреймворки и базы данных [6].

Сообщество и документация: Django имеет активное сообщество, которое организует конференции, форумы и чаты. Официальная документация Django считается одной из лучших в мире разработки благодаря подробным руководствам и примерам.

Преимущества Django:

- Быстрая разработка: Django ускоряет процесс создания приложений благодаря готовым компонентам и принципу DRY, который минимизирует дублирование кода [22].
- Простота обучения: благодаря читаемому синтаксису Python и подробной документации, Django доступен как новичкам, так и опытным разработчикам.
- Высокая безопасность: встроенные механизмы защиты и оперативные обновления делают Django надежным выбором для приложений, работающих с пользовательскими данными [21].
- Масштабируемость: Django подходит для проектов любого масштаба, от стартапов до крупных платформ, благодаря поддержке кэширования, балансировки нагрузки и микросервисной архитектуры.
- Большое сообщество: активное сообщество, множество туториалов, книг и готовых решений упрощают разработку и устранение ошибок.
- Гибкость: Django работает с любыми клиентскими фреймворками и поддерживает доставку контента в различных форматах.

Недостатки Django:

- Избыточность для небольших проектов: Django может быть слишком громоздким для простых приложений, где более легкие фреймворки, такие как Flask, будут предпочтительнее.

- **Монотонность:** Django является «мнениеобразующим» фреймворком, навязывающим определенные подходы к разработке. Это может ограничивать гибкость в некоторых сценариях.
- **Крутая кривая обучения для сложных функций:** хотя базовые возможности Django просты, освоение продвинутых функций, таких как кастомизация ORM или настройка DRF, требует времени и опыта.
- **Зависимость от внутренних компонентов:** некоторые встроенные приложения тесно связаны с ORM, что усложняет их замену на альтернативные решения.
- **Производительность:** хотя Django достаточно быстр для большинства приложений, он может уступать более легковесным фреймворкам в сценариях, требующих экстремально высокой скорости отклика.

Области применения Django:

- **Социальные сети:** Instagram и Pinterest используют Django для управления пользовательскими данными и медиа.
- **Системы управления контентом:** Django идеален для создания блогов, новостных сайтов и вики благодаря встроенной админ-панели.
- **Электронная коммерция:** поддержка платежных систем и каталогов делает Django подходящим для интернет-магазинов.
- **Аналитика и обработка данных:** Django часто используется в научных и аналитических приложениях благодаря интеграции с библиотеками Python, такими как NumPy и Pandas.
- **API и мобильные бэкенды:** Django REST Framework позволяет создавать API для мобильных приложений.
- **Географические информационные системы:** модуль GeoDjango поддерживает разработку приложений с пространственными данными.

2.3.2 Сравнение с другими фреймворками

Django и Flask: Flask – более легковесный фреймворк, подходящий для небольших проектов, но требует подключения сторонних библиотек для

большинства функций, которые в Django встроены. Django лучше для крупных приложений, где важна масштабируемость и безопасность [8].

Django и FastAPI: FastAPI превосходит Django в производительности и подходит для асинхронных API, но уступает в универсальности и наличии готовых решений, таких как админ-панель.

Django и Ruby on Rails: Оба фреймворка следуют принципу DRY и предлагают похожий функционал. Выбор между ними часто зависит от предпочтения языка.

Django – это мощный и универсальный фреймворк, который идеально подходит для разработки сложных веб-приложений благодаря своей философии «всё включено», высокой безопасности и поддержке сообщества. Его преимущества включают быструю разработку, масштабируемость и обширную экосистему, что делает его выбором для стартапов, крупных компаний и индивидуальных разработчиков. Однако для небольших проектов или приложений, требующих максимальной производительности, могут быть предпочтительны более легкие альтернативы, такие как Flask или FastAPI.

2.4 Сравнение СУБД

В настоящий момент существуют различные реляционные СУБД, которые обеспечивают разный функционал. Для изучения их преимуществ и недостатков будет проведено сравнение самых популярных СУБД для более наглядного понимания, предоставляемых ими, возможностей.

PostgreSQL – объектно-реляционная СУБД, известная своей мощностью, расширяемостью и строгим соответствием стандартам SQL. Она поддерживает сложные запросы, расширенные типы данных и подходит для высоконагруженных систем.

MySQL – реляционная СУБД, ориентированная на простоту, скорость и надежность, особенно для веб-приложений. Она популярна благодаря легкости настройки и интеграции с веб-фреймворками.

SQLite – легковесная встраиваемая СУБД, не требующая серверного процесса. Данные хранятся в одном файле, что делает её идеальной для приложений с ограниченными ресурсами.

Архитектура влияет на сценарии использования. PostgreSQL и MySQL – клиент-серверные, поддерживают параллельные подключения, хранят данные в каталогах или базах. SQLite – встраиваемая, данные в одном файле, с ограниченным параллелизмом. PostgreSQL и MySQL подходят для сетевых приложений, SQLite – для автономных.

Производительность зависит от типа запросов. PostgreSQL excels в аналитике (сложные JOIN-ы, оконные функции), но медленнее для простых операций. MySQL быстра для CRUD-запросов, идеальна для веб-приложений. SQLite быстра в одном потоке, но не справляется с множественными записями. PostgreSQL – для BI, MySQL – для магазинов, SQLite – для мобильных приложений.

Функциональность определяет гибкость. PostgreSQL поддерживает JSON/JSONB, массивы, геоданные (PostGIS), ACID-транзакции, PL/pgSQL. MySQL предлагает JSON и базовые функции, но менее гибка. SQLite ограничена базовыми типами (INTEGER, TEXT), без JSON или хранимых процедур. PostgreSQL – самая функциональная, MySQL – стандартная, SQLite – минималистичная.

Масштабируемость важна для роста нагрузки. PostgreSQL масштабируется вертикально и через репликацию (Citius для шардинга), подходит для высоких нагрузок (Instagram). MySQL поддерживает простую репликацию (master-slave), эффективна для веб-приложений. SQLite не масштабируется, ограничена одним устройством. PostgreSQL и MySQL – для серверов, SQLite – для локальных задач.

Безопасность защищает данные. PostgreSQL предлагает роли, Row-Level Security, SSL, Kerberos. MySQL поддерживает SSL и базовые привилегии, менее гибка. SQLite полагается на файловую систему, шифрование через

SQLCipher. PostgreSQL – самая безопасная, MySQL – умеренная, SQLite – минимальная.

Сообщество влияет на ресурсы. PostgreSQL имеет активное сообщество, лучшую документацию, поддержку от EDB. MySQL – большое сообщество, поддержка Oracle, менее структурированная документация. SQLite – небольшое сообщество, ясная документация. PostgreSQL лидирует по документации, MySQL – по популярности, SQLite – по простоте.

Применение в различных областях зависит от задач. PostgreSQL – для сложных веб-приложений, аналитики, GIS. MySQL – для CMS, e-commerce. SQLite – для мобильных приложений, прототипов.

На основании рассмотренной ранее информации можно сделать краткую сводку об особенностях выбранных СУБД (Таблица 2.1).

Таблица 2.1 – Таблица сравнения СУБД

Критерий	PostgreSQL	MySQL	SQLite
Тип	Клиент-сервер	Клиент-сервер	Встраиваемая
Производительность	Высокая для аналитики	Высокая для CRUD	Высокая для одиночных операций
Функциональность	Богатая (JSON, геоданные, CTE)	Умеренная (JSON, базовые функции)	Базовая
Масштабируемость	Хорошая (репликация, шардинг)	Хорошая (репликация)	Отсутствует
Безопасность	Высокая (роли, SSL, RLS)	Умеренная (SSL, привилегии)	Минимальная (файловая)
Размер и ресурсы	Требует сервера, ресурсоемкая	Требует сервера, умеренные ресурсы	Легковесная, минимальные ресурсы
Сообщество	Активное, отличная документация	Большое, хорошая документация	Небольшое, ясная документация
Лучшее применение	Сложные веб-приложения, аналитика	Веб-приложения, CMS	Мобильные, прототипы

Подведем итоги по сравнению СУБД. PostgreSQL – лучший выбор для сложных, высоконагруженных приложений, аналитических систем и проектов, где важна гибкость (например, JSON, геоданные). Подходит для Django, Ruby on Rails и других современных фреймворков.

MySQL – оптимальна для веб-приложений с большим количеством транзакций, где важна простота настройки и высокая скорость для стандартных операций. Популярна в связке с PHP (например, WordPress).

SQLite – идеальна для мобильных приложений, встраиваемых систем и прототипирования, где требуется минимальный размер и отсутствие серверной инфраструктуры.

2.5 Изучение SaaS модели

SaaS (Software as a Service) – это модель предоставления программного обеспечения, при которой приложения размещаются в облаке и доставляются пользователям через интернет по подписке. Вместо покупки и установки программ на локальные устройства пользователи получают доступ к ним через веб-браузер или API, оплачивая регулярную (обычно ежемесячную или ежегодную) подписку. SaaS является одной из трех основных моделей облачных вычислений, наряду с IaaS (Infrastructure as a Service) и PaaS (Platform as a Service) [10].

Ключевые характеристики SaaS модели:

- Облачная доставка: программное обеспечение размещается на серверах поставщика и доступно через интернет. Пользователям не нужно устанавливать приложение на свои устройства. Доступ осуществляется через веб-браузер или мобильное приложение.
- Модель подписки: пользователи платят регулярную абонентскую плату вместо единовременной покупки.
- Централизованное управление: поставщик SaaS отвечает за обслуживание, обновления, исправление ошибок и управление

инфраструктурой. Пользователи получают всегда актуальную версию ПО без необходимости ручного обновления.

- Кроссплатформенность: SaaS-приложения работают на любых устройствах с доступом в интернет, что обеспечивает гибкость использования.
- Автоматическое масштабирование: поставщик может динамически увеличивать ресурсы для обработки роста числа пользователей или нагрузки.

Преимущества использования SaaS модели:

- Доступность и удобство: доступ к приложению из любой точки мира с интернет-соединением. Не требует установки или настройки на стороне пользователя.
- Экономия затрат: нет необходимости в покупке дорогих лицензий или серверов. Модель подписки позволяет платить только за используемые функции.
- Автоматические обновления: поставщик регулярно обновляет ПО, добавляя новые функции и исправляя уязвимости, без участия пользователя.
- Масштабируемость: легко увеличить количество пользователей или функциональность, просто изменив тарифный план.
- Кроссплатформенность: работает на Windows, macOS, Linux, iOS, Android, что упрощает доступ для разных устройств.
- Снижение нагрузки на IT-отдел: поставщик берет на себя обслуживание, резервное копирование и мониторинг, освобождая внутренние ресурсы компании.
- Быстрое внедрение: развертывание занимает меньше времени, так как не требуется установка или настройка серверов.

Недостатки использования SaaS модели:

- Зависимость от интернета: без стабильного интернет-соединения доступ к приложению невозможен.
- Ограниченная кастомизация: SaaS-приложения часто имеют фиксированный функционал, что может не подходить для специфических требований.

- Зависимость от поставщика: пользователи привязаны к вендору, что создает сложности перехода на другое ПО.
- Вопросы безопасности и конфиденциальности: данные хранятся на серверах поставщика, что вызывает опасения у компаний, работающих с чувствительной информацией.
- Долгосрочные затраты: подписка может стать дороже, чем единовременная покупка ПО, особенно для долгосрочного использования.
- Ограниченный контроль: пользователи не управляют инфраструктурой, что может быть проблемой для компаний, требующих полного контроля над серверами.

SaaS – это гибкая и экономичная модель доставки программного обеспечения, которая идеально подходит для компаний и пользователей, стремящихся минимизировать затраты на инфраструктуру и обслуживание. Она обеспечивает быстрый доступ к современным приложениям, автоматические обновления и масштабируемость, но сопряжена с зависимостью от интернета, ограниченной кастомизацией и потенциальными рисками безопасности. SaaS применяется в самых разных областях – от офисных приложений до сложных CRM-систем, и её популярность продолжает расти благодаря удобству и доступности.

3 СИСТЕМНЫЙ АНАЛИЗ СЕРВЕРНОЙ ЧАСТИ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ЗАМЕТОК

3.1 Определение объекта и предмета анализа

Объект исследования – это серверная часть приложения заметок. Серверная часть представляет собой техническую систему, которая отвечает за хранение, обработку и передачу данных в приложении заметок, предназначенном для образовательных целей. Она включает в себя такие элементы, как программное обеспечение, базы данных, серверное оборудование и сетевые компоненты, которые вместе обеспечивают работу приложения.

Предмет исследования – это конкретные аспекты или характеристики объекта, которые мы хотим изучить, разработать или улучшить. Для данной темы предметом исследования могут быть следующие направления:

- Архитектура серверной части: структура системы, используемые компоненты и их взаимодействие.
- Производительность: способность системы справляться с большим количеством запросов, скорость отклика и возможности масштабирования.
- Безопасность: защита данных пользователей, шифрование и предотвращение несанкционированного доступа.
- Интеграция с клиентской частью: взаимодействие серверной части с интерфейсом приложения и используемые протоколы обмена данными.
- Функциональность: поддержка операций с заметками, таких как создание, редактирование, удаление и совместное использование.
- Адаптация к образовательным целям: специальные функции для учебного процесса, например, совместная работа, отслеживание прогресса.

3.2 Функции и структура исследуемого объекта.

Конечной целью исследования является проведение системного анализа серверной части веб-приложения заметок для обеспечения понимания функциональных аспектов и структуры объекта исследования, что требуется для определения требований разрабатываемой системы и предупреждения возможных ошибок во время разработки.

Для выполнения цели, следует определить будущий функционал и структуру серверной части веб-приложения заметок, для достижения которых будут составлены требования.

Серверная часть веб-приложения заметок должна иметь следующие функции [11]:

Аутентификация: пользователь должен иметь возможность зарегистрироваться и войти в свой аккаунт для просмотра доступных ему заметок.

CRUD операции: пользователь должен иметь возможность создавать, читать, редактировать и удалять свои заметки.

Теги и категории: пользователь должен иметь возможность добавлять теги, присваивать их заметкам, а также создавать категории, в которые можно будет добавлять заметки. Все это поможет структурировать заметки.

Ежедневник: пользователь должен иметь возможность добавлять заметки на определенные даты календаря.

Совместное редактирование: пользователь должен иметь возможность дать доступ к выбранным заметкам или получить доступ к заметкам других пользователей, которые поделились ими с ним, для чтения, либо их редактирования.

Поиск и фильтрация: пользователь должен иметь возможность искать заметки, по ключевым словам, и фильтровать заметки по дате написания, тегам и названию.

Форматирование текста: пользователь должен иметь возможность форматировать текст.

Экспорт текста: пользователь должен иметь возможность экспортировать заметку в текстовый формат файла.

Структура стандартной серверной части приложений на фреймворке Django следует его философии MVT и принципа модульности.

Философия MVT предполагает использование разделение архитектуры веб-приложения на 3 части:

1. Model: управляет структурой и взаимодействием данных в приложении Django, что делает их основой приложений Django. Модели Django используют мощную функцию, которую называют объектно-реляционным отображением (ORM), которая устраняет разрыв между реляционной базой данных и кодом на Python. Он преобразует объекты (классы) Python в таблицы базы данных, их атрибуты – в столбцы, а экземпляры – в строки внутри этих таблиц. Одним из огромных преимуществ ORM является то, что он позволяет вам взаимодействовать с базой данных, используя объекты Python, вместо написания SQL-запросов. Модели Django инкапсулируют всю логику, связанную с базой данных, и определяют структуру базы данных, действуя как схема для данных.

2. View: отвечает за обработку запросов пользователей и возврат ответов. Они служат связующим звеном между models и template, собирая данные из объектов модели, выполняя над ними логические операции, а затем передавая данные для отображения. Могут быть написаны как функции или на основе классов, в зависимости от сложности и требований вашего приложения.

3. Template: отвечает за вывод окончательного HTML-кода в браузере. Они определяют, как должны быть представлены данные, используя комбинацию HTML и языка шаблонов Django. В этом слое вы можете получать доступ к переменным, определенным в ваших представлениях, и использовать управляющие структуры в вашем шаблоне.

Взаимодействие элементов такой архитектуры показано на рисунке 3.1.

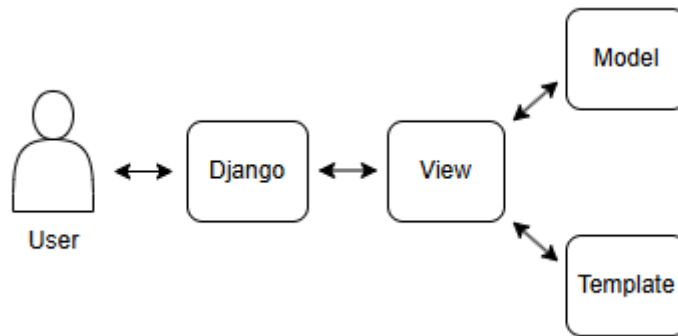


Рисунок 3.1 – MVT архитектура веб-приложения

Однако в разрабатываемой серверной части предполагается использование измененной архитектуры для использования фреймворка React. Такая архитектура называется SPA и предполагает использование одной динамически обновляемой страницы веб-приложения. Обновленная архитектура показана на рисунке 3.2.

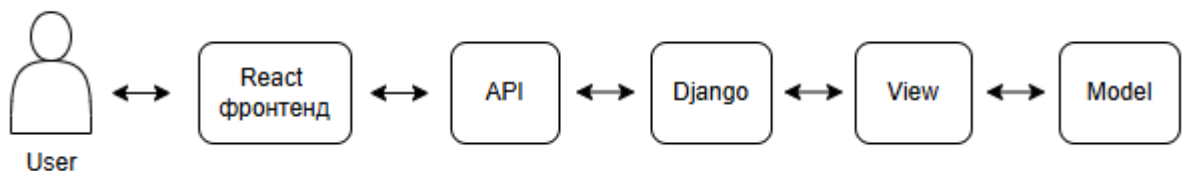


Рисунок 3.2 – Архитектура разрабатываемого веб-приложения

3.2.1 Описание архитектуры веб-приложения

Модели данных (Models):

1) Tag (Тег) имеет поля name и user, связана с моделью Note типом связи многие к многим.

2) Folder (Папка) имеет поля name, user, parent, created_at, updated_at, связана внешним ключом с моделью User, связью одни к многим с моделью User, также имеет связь сама с собой для создания иерархии папок.

3) Note (Заметка) имеет поля title, content, user, folder, created_at, updated_at, связана с моделями User и Folder внешним ключом, имеет связь вида многие к многим с моделью Tag

4) User (Пользователь) является стандартной моделью фреймворка.

Описание архитектуры API: использует JWT (JSON Web Tokens) через Django Rest Framework SimpleJWT

Эндпоинты:

- /api/auth/register/ - регистрация
- /api/auth/login/ - получение токена
- /api/auth/refresh/ - обновление токена
- /api/auth/profile/ - управление профилем

Основное API (notes_api): использует ViewSets для REST API:

1. TagViewSet:

- 1.1 CRUD операции для тегов.
- 1.2 Фильтрация по пользователю.

2. FolderViewSet:

- 2.1 CRUD операции для папок.
- 2.2 Поддержка иерархической структуры.
- 2.3 Фильтрация по родительской папке.

3. NoteViewSet:

- 3.1 CRUD операции для заметок.
- 3.2 Расширенная фильтрация через NoteFilter.
- 3.3 Поддержка тегов и папок.

4. Специальные эндпоинты:

- 4.1 /api/structure/ - получение структуры папок.
- 4.2 /api/sidebar/ - данные для боковой панели.

Сериализация: это процесс преобразования структуры данных или состояния объекта в стандартный формат обмена данными, который может быть сохранен или передан и восстановлен позже. Когда результирующая последовательность битов перечитывается в соответствии с форматом сериализации, ее можно использовать для создания семантически идентичного клона исходного объекта.

Используемые сериализации:

- TagSerializer - сериализация тегов.
- FolderSerializer - сериализация папок.
- NoteSerializer - сериализация заметок с поддержкой тегов.
- FolderStructureSerializer - рекурсивная сериализация структуры папок.
- SidebarSerializer - сериализация данных для боковой панели.

Фильтрация и поиск использует django-filter для расширенной фильтрации и поддерживает:

- Поиск по заголовку и содержимому.
- Фильтрацию по тегам.
- Фильтрацию по дате создания.
- Фильтрацию по папкам.

Безопасность:

- JWT аутентификация.
- CORS настройки для безопасного взаимодействия с фронтендом.
- Проверка прав доступа через IsAuthenticated.
- Валидация данных через сериализаторы.

Конфигурация веб приложения находится в settings.py и включает:

- REST Framework конфигурацию.
- JWT настройки.
- CORS настройки.
- Настройки базы данных (SQLite в разработке).
- Установленные приложения и middleware.

Особенности реализации:

- Многоуровневая структура папок.
- Система тегов с уникальностью для каждого пользователя.
- Оптимизированные запросы к базе данных.
- Поддержка фильтрации и поиска.
- Автоматическое отслеживание времени создания/обновления.

Эта архитектура обеспечивает:

- Масштабируемость.
- Безопасность.
- Производительность.
- Удобство разработки.
- Гибкость в расширении функционала.

3.3 Выделение подсистем в объекте исследования

В рамках работы был проведен системный анализ, используя методологию IDEF0. IDEF представляет собой нотацию, разработанную для функционального моделирования в системах. Разработанная диаграмма нулевого уровня, выделяет главные составляющие и определяет их взаимосвязи. Нулевой уровень диаграммы представлен на рисунке 3.3.

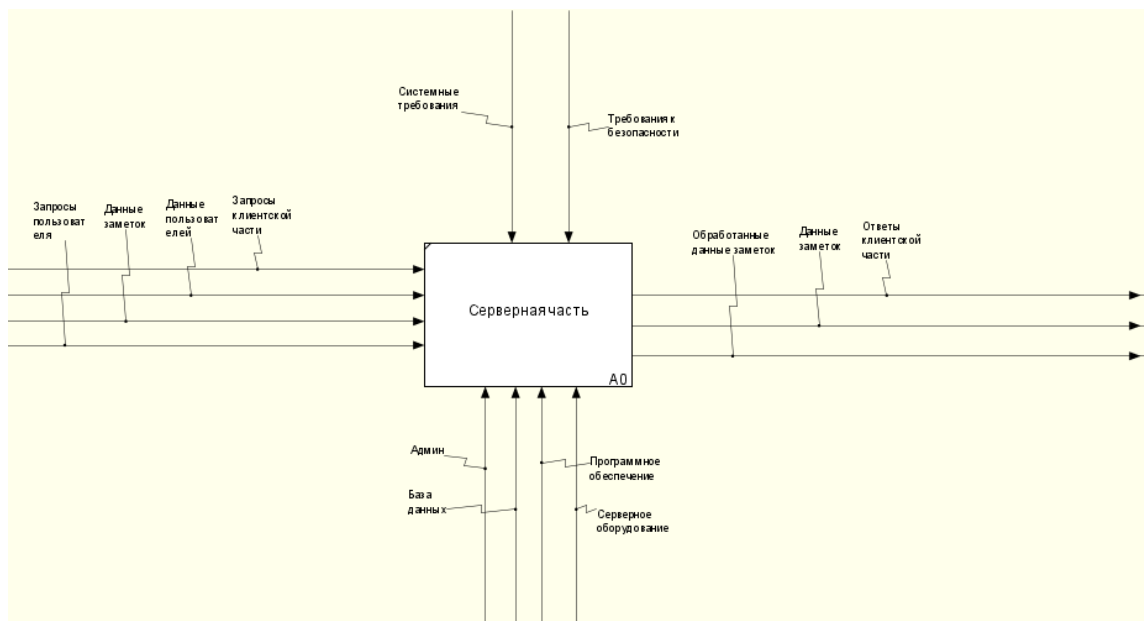


Рисунок 3.3 – Диаграмма в нотации IDEF0 для серверной части веб-приложения

В серверной части выделяются следующие подсистемы:

- Подсистема аутентификации и управления пользователями
- Подсистема управления заметками

- Подсистема управления папками
- Подсистема управления тегами
- Подсистема организации структуры данных
- Подсистема обработки запросов и безопасности

Разберем каждую подсистему, включая их компоненты, функции и роли.

3.3.1 Подсистема аутентификации и управления пользователями

Эта подсистема отвечает за регистрацию, вход, выход, управление профилями пользователей и обеспечение аутентификации для доступа к защищенным ресурсам. Она гарантирует, что только зарегистрированные пользователи могут работать с заметками, папками и тегами, а их действия ограничены данными, к которым у них есть доступ.

Файлы проекта:

- authentication/models.py: модель пользователя.
- authentication/serializers.py: сериализаторы для обработки данных регистрации, входа и профиля.
- authentication/views.py: логика обработки запросов для регистрации, входа, выхода, обновления токенов и получения профиля.
- notes_project/settings.py: настройки аутентификации (JWT, CORS).
- notes_project/urls.py: маршруты для эндпоинтов аутентификации.

Модели данных:

User: стандартная модель Django с полями username, email, password и связями с заметками, папками и тегами через ForeignKey.

API Endpoints:

- POST /api/auth/register/: регистрация нового пользователя.
- POST /api/auth/login/: вход и выдача токенов (access и refresh).
- POST /api/auth/refresh/: обновление токена.
- GET /api/auth/profile/: получение данных профиля.
- POST /api/auth/logout/: выход и инвалидация токенов.

Middleware:

- JWT-аутентификация для проверки токенов на защищенных эндпоинтах.
- Обработка разрешений для ограничения доступа к данным пользователя.

Взаимодействие с другими системами:

- С другими подсистемами: предоставляет данные пользователя для подсистем управления заметками, папками и тегами, чтобы ограничить доступ только к собственным данным.
- С клиентской частью: передаёт токены и данные профиля для авторизации на фронтенде.

3.3.2 Подсистема управления заметками

Она отвечает за создание, чтение, обновление, удаление и фильтрацию заметок. Она является ядром приложения, так как заметки – основной инструмент для студентов и преподавателей в образовательном процессе.

Файлы проекта:

- `notes_api/models.py`: модель `Note` с полями `title`, `content`, `created_at`, `updated_at`, `user`, `folder`, `tags`.
- `notes_api/serializers.py`: сериализаторы для преобразования данных заметок в JSON.
- `notes_api/views.py`: логика обработки запросов для работы с заметками.
- `notes_api/urls.py`: маршруты для эндпоинтов заметок.

Модели данных:

`Note`: основная модель с полями `title`(заголовок заметки), `content`(текст), `created_at`, `updated_at`(метки времени), `user`(связь с пользователем), `folder`(связь с папкой), `tags`(связь с тегами).

API Endpoints:

- `GET /api/notes/`: получение списка заметок пользователя с фильтрами (по тегам, папкам, дате).

- POST /api/notes/: создание новой заметки.
- GET /api/notes/{id}/: получение конкретной заметки.
- PUT /api/notes/{id}/: обновление заметки.
- DELETE /api/notes/{id}/: удаление заметки.

Взаимодействие с другими системами:

- С подсистемой аутентификации: проверяет, что пользователь имеет доступ только к своим заметкам.
- С подсистемами папок и тегов: связывает заметки с папками (folder) и тегами (tags) для организации данных.
- С клиентской частью: передает данные заметок в формате JSON для отображения в веб-приложении.

3.3.3 Подсистема управления папками

Она отвечает за создание, чтение, обновление и удаление папок, а также поддержку иерархической организации заметок. Папки помогают пользователям структурировать заметки, что особенно важно в образовательном контексте для разделения материалов по курсам или темам.

Файлы проекта:

- notes_api/models.py: модель Folder с полями name, user, parent.
- notes_api/serializers.py: сериализаторы для преобразования данных папок.
- notes_api/views.py: логика обработки запросов для работы с папками.
- notes_api/urls.py: маршруты для эндпоинтов папок.

Модели данных:

Folder: имеет поля name (название папки), user (связь с пользователем), parent (самоссылка для вложенных папок).

API Endpoints:

- GET /api/folders/: получение списка папок пользователя.
- POST /api/folders/: создание новой папки.

- GET /api/folders/{id}/: получение конкретной папки.
- PUT /api/folders/{id}/: обновление папки.
- DELETE /api/folders/{id}/: удаление папки.

Взаимодействие с другими системами:

- С подсистемой аутентификации: ограничивает доступ к папкам только их владельцу.
- С подсистемой заметок: связывает заметки с папками через поле folder в модели Note.
- С клиентской частью: передаёт данные папок для отображения в боковом меню или дереве папок.

3.3.4 Подсистема управления тегами

Отвечает за создание, чтение, обновление и удаление тегов, а также их привязку к заметкам. Теги помогают отмечать заметки, что упрощает их поиск и анализ.

Файлы проекта:

- notes_api/models.py: модель Tag с полями name, user, color.
- notes_api/serializers.py: сериализаторы для преобразования тегов.
- notes_api/views.py: логика обработки запросов для работы с тегами.

Модели данных:

Tag: имеет поля name(название тега), user(связь с пользователем).

API Endpoints:

- GET /api/tags/: получение списка тегов пользователя.
- POST /api/tags/: создание нового тега.
- GET /api/tags/{id}/: получение конкретного тега.
- PUT /api/tags/{id}/: обновление тега.
- DELETE /api/tags/{id}/: удаление тега.

Взаимодействие с другими системами:

- С подсистемой аутентификации: ограничивает доступ к тегам только их владельцу.

- С подсистемой заметок: связывает теги с заметками через поле tags в модели Note.
- С клиентской частью: передаёт данные тегов для фильтрации заметок или визуального оформления.

3.3.5 Подсистема организации структуры данных

Эта подсистема отвечает за предоставление иерархической структуры папок и заметок, а также данных для бокового меню. Она помогает клиентской части отображать организованные данные, что критично для навигации в образовательном приложении.

Файлы проекта:

- notes_api/views.py: логика обработки запросов для структуры и бокового меню.
- notes_api/serializers.py: сериализаторы для преобразования данных папок, заметок и тегов в иерархический формат.
- notes_api/urls.py: маршруты для эндпоинтов структуры.

Модели данных:

- Folder: для построения иерархии папок через поле parent.
- Note: для связи заметок с папками и тегами.
- Tag: для включения тегов в структуру.

API Endpoints:

- GET /api/structure/: получение иерархической структуры папок и заметок.
- GET /api/sidebar/: получение данных для бокового меню (список папок, тегов, избранных заметок).

Взаимодействие с другими системами:

- С подсистемами заметок, папок и тегов: собирает данные от этих подсистем для формирования структуры.

- С подсистемой аутентификации: ограничивает структуру только данными текущего пользователя.
- С клиентской частью: передаёт JSON-данные для построения дерева папок или бокового меню.

3.3.6 Подсистема обработки запросов и безопасности

Отвечает за обработку всех входящих запросов, обеспечение безопасности API, управление доступом и взаимодействие с клиентской частью. Она действует как "посредник" между клиентом и другими подсистемами, гарантируя корректную и безопасную работу.

Файлы проекта:

- notes_project/settings.py: настройки CORS, JWT, middleware.
- notes_project/urls.py: главная маршрутизация API.

Middleware для JWT-аутентификации, CORS и проверки permissions.

Модели данных:

Косвенно использует модель User для проверки прав доступа.

API Endpoints:

Все эндпоинты зависят от этой подсистемы для обработки запросов и проверки безопасности.

Middleware:

- JWT-аутентификация: проверяет токены для всех защищенных эндпоинтов.
- CORS: разрешает запросы с фронтенда (например, с домена веб-приложения).
- Permissions: гарантирует, что пользователь имеет доступ только к своим данным (заметкам, папкам, тегам).

Взаимодействие с другими системами:

- С подсистемой аутентификации: получает данные о токенах и правах пользователя.

- С другими подсистемами: передаёт запросы к соответствующим обработчикам (заметки, папки, теги, структура).
- С клиентской частью: принимает HTTP-запросы и возвращает JSON-ответы.

Подсистемы образуют целостную серверную часть приложения, взаимодействуя через:

- API: все подсистемы используют REST API для обмена данными с клиентской частью.
- Базу данных: модели User, Note, Folder, Tag связаны через ForeignKey и ManyToManyField, обеспечивая единое хранилище.
- Middleware: подсистема обработки запросов и безопасности фильтрует все входящие запросы, передавая их соответствующим подсистемам.
- Сериализаторы: преобразуют данные между моделями в JSON.

3.4 Определение связей подсистемам с внешней средой

Внешняя среда включает все, что находится за пределами серверной части, но взаимодействует с ней:

- Клиентская часть: веб-приложение, через которое пользователи (студенты, преподаватели) работают с заметками.
- Пользователи: студенты, преподаватели, администраторы.
- Внешние сервисы: облачные хранилища, системы единого входа, аналитические инструменты.
- Сетевые инфраструктуры: интернет, DNS, облачные платформы.

Внешние связи описывают, как подсистемы взаимодействуют с элементами внешней среды. Рассмотрим каждую подсистему и определим ее связи.

3.4.1 Подсистема аутентификации и управления пользователями

С клиентской частью:

- Связь: передача данных для регистрации, входа, профиля и токенов.

- Тип: двунаправленная (запросы/ответы).
- Механизм: эндпоинты /api/auth/register/, /api/auth/login/, /api/auth/profile/ и др. возвращают JSON с токенами или данными профиля.

С пользователями:

- Связь: получение учетных данных и предоставление доступа.
- Тип: входящая (данные от пользователя), исходящая (токены, профиль).
- Механизм: пользователь вводит данные через интерфейс клиентской части.

С сетевыми инфраструктурами:

- Связь: передача данных через HTTPS.
- Тип: двунаправленная.
- Механизм: запросы проходят через интернет, DNS и облачную платформу.

3.4.2 Подсистема управления заметками

С клиентской частью:

- Связь: передача данных заметок для отображения и редактирования.
- Тип: двунаправленная.
- Механизм: эндпоинты /api/notes/ и /api/notes/{id}/ возвращают JSON с данными заметок.

С пользователями:

- Связь: получение данных заметок (title, content) и предоставление результатов.
- Тип: входящая (ввод данных), исходящая (отображение заметок).
- Механизм: пользователь создает заметку через интерфейс.

С сетевыми инфраструктурами:

- Связь: передача данных через HTTPS.
- Тип: двунаправленная.

- Механизм: запросы и ответы проходят через облачную платформу.

3.4.3 Подсистема управления папками

С клиентской частью:

- Связь: передача данных папок для отображения и управления.
- Тип: двунаправленная.
- Механизм: эндпоинты `/api/folders/` и `/api/folders/{id}/` возвращают

JSON.

С пользователями:

- Связь: получение названий папок и предоставление структуры.
- Тип: входящая (ввод данных), исходящая (отображение).
- Механизм: пользователь создает папку через интерфейс.

С сетевыми инфраструктурами:

- Связь: передача данных через HTTPS.
- Тип: двунаправленная.

3.4.4 Подсистема управления тегами

С клиентской частью:

- Связь: передача данных тегов для фильтрации и отображения.
- Тип: двунаправленная.
- Механизм: эндпоинты `/api/tags/` и `/api/tags/{id}/`.

С пользователями:

- Связь: получение названий и цветов тегов.
- Тип: входящая (ввод), исходящая (отображение).
- Механизм: пользователь создает тег через интерфейс.

С сетевыми инфраструктурами:

- Связь: передача данных через HTTPS.
- Тип: двунаправленная.

3.4.5 Подсистема организации структуры данных

С клиентской частью:

- Связь: передача иерархии для навигации.
- Тип: двунаправленная.
- Механизм: эндпоинты `/api/structure/` и `/api/sidebar/`.

С пользователями:

- Связь: предоставление структуры для навигации.
- Тип: исходящая.
- Механизм: пользователь видит структуру через интерфейс.

С сетевыми инфраструктурами:

- Связь: передача данных через HTTPS.
- Тип: двунаправленная.
- Механизм: запросы и ответы проходят через облачную платформу.

3.4.6 Подсистема обработки запросов и безопасности

С клиентской частью:

- Связь: прием всех HTTP-запросов и отправка ответов.
- Тип: двунаправленная.
- Механизм: Middleware (CORS, JWT) и маршрутизация через `urls.py`.

С пользователями:

- Связь: косвенная, через клиентскую часть.
- Тип: входящая (запросы), исходящая (ответы).
- Механизм: пользователь инициирует запрос через интерфейс.

3.5 Системотехнические принципы

Системотехнические принципы – это фундаментальные правила, которые направляют проектирование и анализ сложных систем, таких как программно-вычислительные системы. Их соблюдение гарантирует, что система будет логичной, управляемой и устойчивой к изменениям.

3.5.1 Принцип единства

Принцип единства требует, чтобы система рассматривалась как целостная структура, где все компоненты (подсистемы) работают вместе для достижения общей цели. Это предполагает согласованность функций, данных и взаимодействий внутри системы и с внешней средой.

Целостность системы: серверная часть приложения заметок рассматривается как единая система, включающая подсистемы аутентификации, заметок, папок, тегов, структуры данных и обработки запросов. Все подсистемы направлены на достижение общей цели – обеспечение надежного и безопасного управления заметками в образовательном процессе.

Согласованность функций:

- Подсистема аутентификации обеспечивает доступ пользователей, что необходимо для работы с заметками, папками и тегами.
- Подсистема заметок предоставляет ядро функциональности (CRUD-операции), поддерживая образовательные задачи, такие как создание конспектов.
- Подсистемы папок и тегов дополняют заметки, обеспечивая их организацию.
- Подсистема структуры данных объединяет данные для навигации, а подсистема обработки запросов гарантирует безопасное взаимодействие.

3.5.2 Принцип декомпозиции

Принцип декомпозиции предполагает разделение сложной системы на более простые подсистемы или компоненты, которые можно анализировать, проектировать и реализовывать независимо, сохраняя их функциональную связь.

Разделение на подсистемы: серверная часть декомпозирована на шесть подсистем, каждая из которых отвечает за отдельный аспект:

- Аутентификация: регистрация, вход, управление профилями.

- Заметки: CRUD-операции и фильтрация.
- Папки: иерархическая организация.
- Теги: категоризация.
- Структура данных: навигация и иерархия.
- Обработка запросов: безопасность и маршрутизация.

Структурная декомпозиция: проект разделен на приложения Django: `notes_api` (заметки, папки, теги, структура) и `authentication` (аутентификация). Каждое приложение содержит свои `models.py`, `serializers.py`, `views.py`, `urls.py`, что упрощает разработку и тестирование.

Функциональная декомпозиция: модель `Note` отделена от `Folder` и `Tag`, что позволяет независимо управлять заметками, их организацией и категоризацией. Эндпоинты API разделены по функциональности: `/api/auth/` для аутентификации, `/api/notes/` для заметок, `/api/folders/` для папок и т.д.

3.5.3 Принцип модульности

Принцип модульности требует, чтобы система состояла из независимых модулей с четко определенными интерфейсами, которые можно разрабатывать, тестировать и заменять без влияния на другие части системы.

Модульная структура проекта: приложения Django (`notes_api`, `authentication`) являются модулями, каждый из которых отвечает за свою функциональность. Файлы внутри приложений также модульны, разделяя данные, логику и маршруты.

Четкие интерфейсы: API endpoints (`/api/notes/`, `/api/auth/`, `/api/folders/`) служат интерфейсами между подсистемами и клиентской частью, используя JSON для обмена данными. Сериализаторы обеспечивают стандартизированное преобразование данных.

Независимость модулей: подсистема аутентификации может быть заменена (например, с JWT на OAuth) без изменения логики заметок. Подсистема заметок работает независимо от папок, так как поле `folder` в `Note` опционально (`null`).

3.5.4 Принцип иерархии

Принцип иерархии предполагает, что система организована в виде уровней, где каждый уровень выполняет свои функции и подчиняется вышестоящему уровню, обеспечивая структурированное управление и взаимодействие.

Иерархическая структура системы:

- Верхний уровень: подсистема обработки запросов и безопасности действует как "управляющий" уровень, фильтруя все запросы через middleware (JWT, CORS, permissions) и маршрутизируя их через `notes_project/urls.py`.
- Средний уровень: подсистемы аутентификации, заметок, папок, тегов и структуры данных выполняют специализированные функции, подчиняясь правилам обработки запросов.
- Нижний уровень: модели данных (User, Note, Folder, Tag) и база данных обеспечивают хранение и доступ к данным.

Иерархия данных:

Модель Folder поддерживает иерархию через поле `parent` (ForeignKey), позволяя создавать вложенные папки. Заметки подчиняются папкам через поле `folder`, а теги связаны с заметками через `ManyToManyField`, создавая иерархию категоризации.

3.5.5 Принцип развития

Принцип развития требует, чтобы система проектировалась с учетом возможности расширения, модификации и адаптации к новым требованиям без значительных изменений в ее структуре.

Расширяемость: модульная структура (приложения `notes_api`, `authentication`) позволяет добавлять новые подсистемы, например, аналитику использования заметок. REST API поддерживает добавление новых эндпоинтов без изменения существующих.

Адаптация к образовательным требованиям: подсистема заметок может быть расширена для поддержки мультимедиа (видео, аудио) путем добавления новых полей в модель Note.

Масштабируемость: использование облачной платформы и контейнеризации обеспечивает горизонтальное масштабирование при росте числа пользователей. Подсистема обработки запросов поддерживает балансировку нагрузки.

Гибкость конфигурации: `notes_project/settings.py` позволяет настраивать параметры (БД, CORS, JWT), что упрощает адаптацию к новым условиям.

3.5.6 Итог системного анализа

Системотехнические принципы обеспечивают структурированный подход к исследованию и разработке серверной части приложения заметок:

Принцип единства: гарантирует целостность системы, согласованность подсистем и их взаимодействие с внешней средой.

Принцип декомпозиции: позволяет разделить систему на подсистемы (аутентификация, заметки, папки и др.), упрощая анализ и разработку.

Принцип модульности: обеспечивает независимость подсистем с четкими интерфейсами (API), что упрощает тестирование и расширение.

Принцип иерархии: структурирует систему по и поддерживает иерархию данных (папки, заметки).

Принцип развития: делает систему гибкой для будущих изменений, таких как новые функции или масштабирование.

4 РАЗРАБОТКА СИСТЕМЫ

4.1 Выбор технологий разработки

Выбор фреймворка Django для разработки серверной части веб-приложения обосновывается его техническими характеристиками, производительностью и соответствием требованиям современных веб-проектов.

Обоснование выбора Django:

- Быстрая разработка и принцип DRY: Django следует принципу «Don't Repeat Yourself» и предоставляет множество встроенных инструментов: ORM для работы с базами данных, систему аутентификации, админ-панель и шаблонизатор. Это сокращает время на реализацию стандартных функций. Например, создание админ-панели занимает минуты, так как она генерируется автоматически на основе моделей данных.
- Масштабируемость и надежность: Django применяется в крупных проектах, что демонстрирует его способность справляться с нагрузками. Фреймворк поддерживает модульную архитектуру, интеграцию с системами кэширования и асинхронными задачами, что упрощает масштабирование.
- Безопасность: Django включает встроенные механизмы защиты от распространенных угроз: SQL-инъекций, XSS (межсайтовый скриптинг), CSRF (межсайтовая подделка запросов) и утечек данных. Например, Django автоматически экранирует данные в шаблонах и предоставляет CSRF-токены для форм, что снижает риск ошибок разработчика.
- Мощная ORM-система: Django ORM позволяет работать с базами данных (PostgreSQL, MySQL, SQLite и др.) через Python-код, абстрагируясь от SQL-запросов. Это упрощает разработку и поддержку, особенно при изменении структуры базы данных. Например, миграции в Django автоматически синхронизируют модели с базой данных.

- Сообщество и экосистема: Django имеет большое сообщество, обширную документацию и множество сторонних библиотек (например, Django REST Framework для API). Это обеспечивает доступ к готовым решениям и поддержке [27].

- Соответствие требованиям проекта: Django идеально подходит для приложений, требующих быстрого прототипирования, работы с реляционными базами данных и стандартных веб-функций (аутентификация, управление контентом).

Аналоги Django и их сравнение:

Для сравнения рассмотрим популярные аналоги Django, используемые для разработки серверной части веб-приложений [9]:

- Flask (Python): легковесный Python-фреймворк, предоставляющий минимальный набор инструментов для создания веб-приложений. Его гибкость и минимализм, подходит для небольших проектов или микросервисов, но требует ручной настройки многих компонентов (например ORM и аутентификации), что увеличивает время разработки по сравнению с Django. Flask лучше для простых API или приложений с нестандартной логикой, но уступает Django в скорости разработки крупных проектов с типичными задачами.

- FastAPI (Python): Современный асинхронный фреймворк для создания высокопроизводительных API. Поддерживает асинхронное программирование, автоматически генерирует документацию и имеет высокая производительность, но менее подходит для монолитных приложений с рендерингом страниц, так как ориентирован на API. Требуется дополнительных библиотек для функционала, аналогичного Django. FastAPI предпочтительнее для асинхронных API с высокой нагрузкой, но Django выигрывает в универсальности и готовых решениях.

- Ruby on Rails (Ruby): фреймворк, схожий с Django, с акцентом на быструю разработку. Он имеет богатая экосистему, встроенные инструменты для работы с базами данных и тестирования. Rails близок к Django по

функционалу, но Python-экосистема и универсальность Django делают его более предпочтительным.

Django выбран для разработки серверной части веб-приложения благодаря сочетанию скорости разработки, безопасности, масштабируемости и мощной экосистемы. Его встроенные инструменты сокращают время на реализацию типичных задач, а поддержка сообщества и библиотек делает его универсальным решением. В сравнении с аналогами, Django предлагает оптимальный баланс между функциональностью, производительностью и простотой поддержки, особенно для монолитных приложений или проектов с реляционными базами данных.

4.2 Основные этапы разработки

Создание веб-приложения на Django, предназначенного для управления заметками, представляет собой процесс, требующий внимания к архитектуре, безопасности и производительности. API для заметок с поддержкой аутентификации пользователей должно быть удобным, масштабируемым и надёжным. В разделе описывается поэтапный подход к разработке такого приложения, охватывающий настройку проекта, моделирование данных, реализацию API и тестирование.

4.2.1 Формирование базовой структуры проекта

Разработка начинается с создания нового Django-проекта. Структура проекта включает стандартные файлы конфигурации, такие как настройки, маршруты и управляющий скрипт. На этом этапе создаются два ключевых приложения: одно для управления аутентификацией и профилями пользователей, другое для функционала основного. Эти приложения отражают разделение ответственности, соответствующее подсистемам серверной части: аутентификация, управление заметками, папками, тегами, организация структуры данных и обработка запросов с обеспечением безопасности.

Конфигурация проекта включает регистрацию созданных приложений и подключение инструментов для разработки REST API, таких как Django REST Framework, который обеспечивает удобную работу с сериализаторами и представлениями. Для поддержки взаимодействия с фронтендом настраивается механизм обработки кросс-доменных запросов (CORS), позволяющий безопасно принимать запросы с клиентской части, например, с веб-приложения на другом домене. Также добавляется поддержка JWT-аутентификации, которая станет основой для подсистемы аутентификации и управления пользователями. JWT обеспечивает безопасную проверку токенов для доступа к защищённым ресурсам, что критично для ограничения пользователей только их собственными данными.

База данных настраивается с использованием SQLite для упрощения разработки, но предусматривается возможность перехода на PostgreSQL, чтобы обеспечить масштабируемость и производительность. В конфигурационных файлах задаются параметры безопасности, такие как middleware для обработки JWT и разрешений. Middleware гарантируют, что все входящие запросы проходят проверку подлинности и авторизации перед передачей в соответствующие подсистемы, такие как управление заметками или папками.

Маршрутизация на уровне проекта настраивается для направления запросов к приложениям аутентификации и заметок. Это создаёт основу для будущих эндпоинтов, таких как регистрация, вход или получение списка заметок.

4.2.2 Построение системы аутентификации

Система аутентификации представляет собой ключевой компонент приложения, обеспечивающий безопасный доступ пользователей к их данным. Она построена с использованием JWT в Django REST Framework и библиотеке SimpleJWT, что позволяет управлять процессами регистрации, входа, обновления токенов. Встроенная модель пользователя Django используется как основа, предоставляя поля, такие как имя пользователя, email и пароль.

На серверной стороне конфигурация включает настройку параметров JWT, таких как сроки действия токенов: короткий срок для access-токена и более длительный для refresh-токена. Это обеспечивает баланс между безопасностью и удобством, минимизируя необходимость частого входа в систему. Middleware для обработки JWT и разрешений интегрируется в проект, гарантируя, что защищённые эндпоинты доступны только авторизованным пользователям, а эндпоинты регистрации остаются открытыми. Настройки кросс-доменных запросов дают доступ только доверенными источниками.

Процесс аутентификации: регистрация начинается с клиентской формы, содержащей поля для email, пароля, подтверждения пароля, имени и фамилии. В серверной части приложения данные проходят проверку: уникальность email, сложность пароля и совпадение паролей. После успешной валидации создаётся пользователь с хешированным паролем, что гарантирует защиту учётных данных.

Вход осуществляется через форму с полями для имени пользователя и пароля. Сервер проверяет данные и выдаёт access- и refresh-токены.

Обновление токена происходит автоматически в серверной части, которая проверяет действительность refresh-токена и генерирует новый access-токен.

Безопасность и защита маршрутов обеспечивается несколькими мерами. Пароли хешируются перед сохранением, глобальная защита маршрутов через проверку авторизации гарантирует, что только аутентифицированные пользователи имеют доступ к заметкам. Эндпоинты регистрации остаются открытыми, чтобы новые пользователи могли создавать учётные записи.

Интеграция и взаимодействие: система аутентификации интегрируется с остальным приложением через защиту всех API-эндпоинтов, требующих авторизации. Серверные компоненты обеспечивают данные пользователя для подсистем управления заметками, папками и тегами, ограничивая доступ только к персональным данным. Клиентская часть передаёт токены и данные профиля для авторизации, а обработка ошибок аутентификации гарантирует

стабильное взаимодействие. Эта архитектура создаёт надёжную основу для безопасного управления пользовательскими сессиями [20].

4.2.3 Моделирование данных для заметок

Функционал заметок строится вокруг трёх ключевых моделей: тегов, папок и заметок. Каждая модель выполняет свою роль в организации данных и поддерживает персонализацию через связь с пользователем. После определения моделей создаются миграции для базы данных.

Модель тегов предназначена для категоризации заметок, позволяя пользователям помечать их для удобного поиска и группировки. Тег представляет собой простую сущность с уникальным именем, связанную с пользователем. Это обеспечивает персонализацию: каждый пользователь работает только со своими тегами. Связь с заметками через отношение многие-ко-многим позволяет применять один тег к нескольким заметкам и наоборот.

Модель папок отвечает за организацию заметок в иерархическую структуру. Папка имеет название и привязана к пользователю. Ключевая особенность модели – поддержка вложенности через рекурсивное отношение, где папка может быть родительской для других папок. Отношение задаётся так, чтобы при удалении родительской папки дочерние элементы обрабатывались корректно, сохраняя целостность данных. Папки также связаны с заметками, что делает модель основой для их структурирования.

Модель заметок является центральной в приложении, представляя основную единицу данных – заметку. Заметка включает заголовок, текстовое содержимое, даты создания и обновления, а также связи с пользователем, папкой и тегами. Связь с пользователем обеспечивает персонализацию, позволяя каждому пользователю видеть свои заметки. Связь с папкой определяет, где хранится заметка. Отношение многие-ко-многим с тегами позволяет присваивать заметке несколько категорий.

Определение моделей завершается созданием миграций, которые синхронизируют структуру базы данных с описанными сущностями. Этот этап

закладывает фундамент для последующей разработки API, обеспечивая надёжную и гибкую структуру данных.

4.2.4 Реализация API для работы с данными

API для заметок представляет собой ядро приложения, обеспечивая функциональность для работы с тегами, папками, заметками и их структурой. Оно строится с использованием Django REST Framework, где сериализаторы преобразуют данные моделей в JSON-формат, подходящий для передачи через HTTP, а представления (ViewSet) обрабатывают запросы. API поддерживает полный набор операций CRUD (создание, чтение, обновление, удаление) для всех сущностей, а также предоставляет специальные эндпоинты для получения иерархической структуры данных и информации для боковой панели.

Эндпоинты аутентификации включают функционал для:

- Регистрации: POST /api/auth/register/
- Входа: POST /api/auth/login/
- Обновления токенов: POST /api/auth/refresh/
- Управления профилем: GET/PUT /api/auth/profile/

Эндпоинт регистрации принимает имя пользователя, email, пароль, подтверждение пароля, имя и фамилию, с валидацией уникальности email и сложности пароля, возвращая данные созданного пользователя. Эндпоинт входа проверяет имя пользователя и пароль, выдавая access- и refresh-токены.

Эндпоинты для тегов позволяют управлять категориями заметок. Пользователи могут:

- Получить список своих тегов: GET /api/tags/
- Создать новый тег: POST /api/tags/
- Просмотреть: GET /api/tags/{id}/
- Обновить: PUT /api/tags/{id}/
- Удалить существующий: DELETE /api/tags/{id}/

Сериализатор для тегов обрабатывает основные поля и автоматически связывает тег с текущим пользователем.

Эндпоинты для папок поддерживают управление иерархической структурой. Пользователи могут:

- Получить список папок: GET /api/folders/
- Создать новую папку: POST /api/folders/
- Просмотреть: GET /api/folders/{id}/
- Обновить: PUT /api/folders/{id}/
- Удалить существующую: DELETE /api/folders/{id}/

Сериализатор для папок включает поля, такие как идентификатор, название, родительская папка и временные метки, с валидацией иерархии для предотвращения некорректных структур. Это обеспечивает корректное отображение вложенных папок в клиентском интерфейсе.

Эндпоинты предоставляют доступ к функционалу для управления основным контентом приложения. Пользователи могут:

- Получить список заметок: GET /api/notes/
- Создать новую заметку: POST /api/notes/
- Просмотреть: GET /api/notes/{id}/
- Обновить: PUT /api/notes/{id}/
- Удалить существующую: DELETE /api/notes/{id}/

Сериализатор для заметок обрабатывает идентификатор, заголовок, содержимое, папку, теги и временные метки, поддерживая вложенную сериализацию тегов и указание тегов через их идентификаторы или названия. Это упрощает работу с категориями и обеспечивает удобное представление данных для фронтенда.

Специальные эндпоинты расширяют функциональность API, предоставляя данные для структурированного отображения:

– GET /api/structure/ – эндпоинт структуры возвращает иерархическую организацию папок с вложенными заметками, используя рекурсивный сериализатор для корректного представления вложенности.

– GET /api/sidebar/ – эндпоинт боковой панели предоставляет данные для клиентского интерфейса, включая корневые папки и список последних заметок, что упрощает навигацию и быстрый доступ к актуальному контенту.

API отличается гибкостью и безопасностью. Все эндпоинты, кроме регистрации и входа, защищены JWT-аутентификацией, а фильтрация по пользователю гарантирует, что данные доступны только их владельцу. Настройки CORS обеспечивают безопасное взаимодействие с фронтендом, ограничивая доступ доверенными источниками.

Оптимизация запросов достигается за счёт использования методов предварительной загрузки данных, таких как `select_related` и `prefetch_related`, что сокращает количество обращений к базе данных. Сериализаторы обрабатывают данные, минимизируя объём передаваемой информации. Валидация данных на уровне сериализаторов предотвращает некорректные запросы.

4.2.5 Организация маршрутизации

Маршрутизация в проекте организована с использованием Django REST Framework и его инструмента `DefaultRouter`, который упрощает создание RESTful-эндпоинтов через `ViewSet`s.

На уровне проекта задаются основные URL-префиксы, направляющие запросы к различным частям приложения. Административный интерфейс доступен через префикс для админ-панели, обеспечивая доступ к управлению данными. Эндпоинты API группируются под общим префиксом, который включает маршруты для приложения заметок, охватывающего теги, папки, заметки и структуру данных. Эндпоинты аутентификации выделены в отдельный префикс, связанный с приложением для управления пользователями, что отражает разделение ответственности между подсистемами.

Такая организация маршрутов гарантирует интуитивно понятную структуру API, где каждый префикс соответствует определённой подсистеме. Использование `DefaultRouter` упрощает добавление новых эндпоинтов, а

разделение маршрутов по приложениям поддерживает модульность. Безопасность маршрутов обеспечивается через глобальную JWT-аутентификацию, интегрированную на уровне middleware, что защищает доступ к данным и упрощает управление правами.

4.2.6 Настройка административного интерфейса

Административный интерфейс Django адаптируется для удобного управления данными. Модели тегов, папок и заметок регистрируются в админ-панели с настройкой отображения, фильтрации и поиска. Это облегчает модерацию и отладку данных в процессе разработки и эксплуатации.

4.2.7 Тестирование и оптимизация

Тестирование охватывает проверку всех эндпоинтов API, включая операции создания, чтения, обновления и удаления. Проверяется корректность фильтрации, прав доступа и бизнес-логики, например, иерархии папок. Валидация данных гарантирует, что API отклоняет некорректные запросы.

Для повышения производительности оптимизируются запросы к базе данных с использованием методов предварительной загрузки данных. При необходимости внедряется кэширование для часто запрашиваемых ресурсов.

Ключевые аспекты разработки:

1. **Безопасность:** аутентификация через JWT обеспечивает надёжный контроль доступа. Данные фильтруются по пользователю, а валидация предотвращает некорректные запросы. Настройки кросс-доменных запросов ограничивают доступ только доверенными источниками.

2. **Производительность:** оптимизация запросов и кэширование сокращают время ответа API. Гибкая фильтрация позволяет пользователям быстро находить нужные данные.

3. **Масштабируемость:** модульная структура и чёткое разделение ответственности упрощают добавление новых функций. Архитектура

поддерживает расширение, например, добавление новых моделей или эндпоинтов.

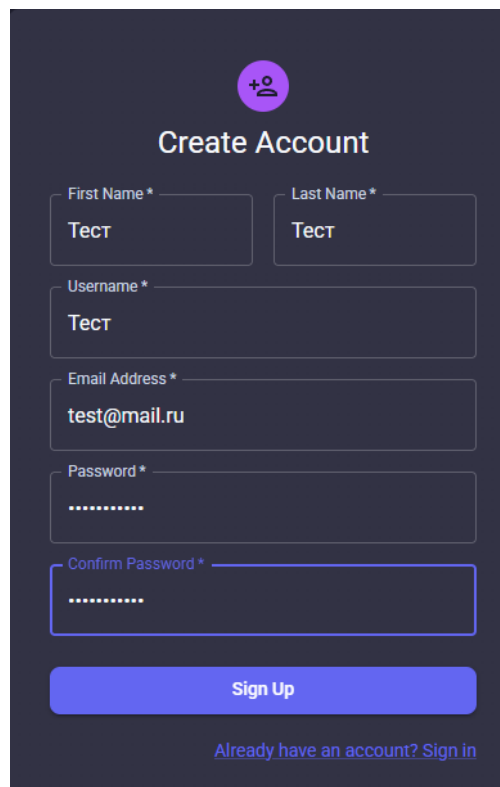
4. Поддерживаемость: чистая архитектура и документированный код облегчают сопровождение. Тестируемость кода позволяет быстро выявлять и исправлять ошибки [16].

Поэтапный подход к разработке Django-приложения для управления заметками обеспечивает логичную последовательность создания компонентов, начиная с базовой настройки и заканчивая готовым продуктом. Чёткое разделение ответственности, возможность тестирования на каждом этапе и акцент на безопасность делают систему надёжной, масштабируемой и удобной для пользователей. Такой процесс создаёт прочную основу для дальнейшего развития приложения, позволяя легко добавлять новые функции и адаптировать его под растущие потребности.

5 ТЕСТИРОВАНИЕ СИСТЕМЫ

5.1 Тестирование аутентификации

Регистрация должна отправлять запрос на создание новой записи в базу данных и шифровать пароль. Ожидается запрос к эндпоинту регистрации и создание нового пользователя. Окно регистрации показано на рисунке 5.1.



The image shows a 'Create Account' registration form on a dark background. At the top is a purple circle icon with a white plus sign and a person silhouette. Below it is the title 'Create Account'. The form contains several input fields: 'First Name *' with 'Тест', 'Last Name *' with 'Тест', 'Username *' with 'Тест', 'Email Address *' with 'test@mail.ru', 'Password *' with masked dots, and 'Confirm Password *' with masked dots. A blue 'Sign Up' button is at the bottom, and a link 'Already have an account? Sign in' is below it.

Рисунок 5.1 – окно регистрации

После входа на сервер был получен запрос на создание нового пользователя: "POST /api/auth/register/ HTTP/1.1" 201 93. В базе данных должна появиться запись о нем. Проверим базу данных через встроенную админ-панель Django (Рисунок 5.2).

Change user

Тест

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password: **algorithm:** pbkdf2_sha256 **iterations:** 870000 **salt:** 2oPJ6w***** **hash:** eP0Wus*****
[Reset password](#)
Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Last name:

Email address:

Рисунок 5.2 – Запись о новом пользователе

Запись была создана и хранит всю указанную информацию без отклонений. Пароль зашифрован и получить к нему доступ через админ-панель не представляется возможным.

Далее проверим запросы, которые получает сервер при входе в аккаунт:

[19/May/2025 02:04:02] "POST /api/auth/login/ HTTP/1.1" 200 483

[19/May/2025 02:04:02] "GET /api/structure/ HTTP/1.1" 200 2

[19/May/2025 02:04:02] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 02:04:02] "GET /api/structure/ HTTP/1.1" 200 2

[19/May/2025 02:04:02] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 02:04:02] "GET /api/sidebar/ HTTP/1.1" 200 61

[19/May/2025 02:04:02] "GET /api/notes/ HTTP/1.1" 200 2

[19/May/2025 02:04:02] "GET /api/sidebar/ HTTP/1.1" 200 61

[19/May/2025 02:04:02] "GET /api/notes/ HTTP/1.1" 200 2

Сервер получает запрос на вход и при подтверждении его валидности одобряет остальные запросы на получение данных. При неправильности введённых данных отвечает ошибкой 401, которая означает неверные учетные данные: [19/May/2025 02:22:03] "POST /api/auth/login/ HTTP/1.1" 401 63

5.2 Тестирование заметок

Для тестирования заметок создадим новую заметку без тегов и принадлежности к папкам. При открытии формы создания заметки (рисунок 5.3), сервер также получает запросы на получение информации о существующих папках, тегах и информации для боковой панели:

[19/May/2025 02:35:31] "GET /api/tags/ HTTP/1.1" 200 2

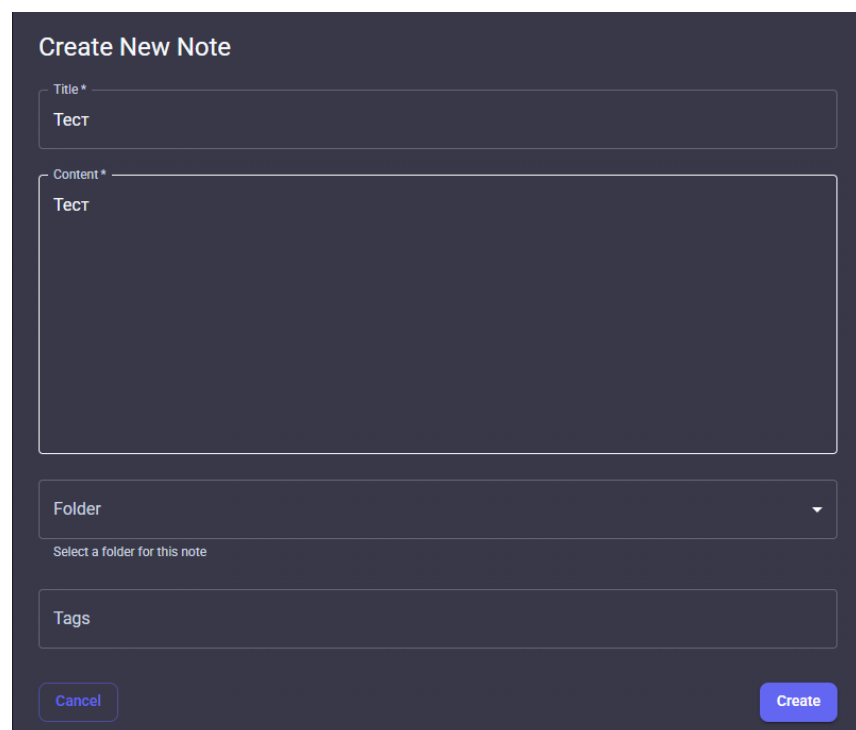
[19/May/2025 02:35:31] "GET /api/structure/ HTTP/1.1" 200 2

[19/May/2025 02:35:31] "GET /api/folders/ HTTP/1.1" 200 2

[19/May/2025 02:35:31] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 02:35:31] "GET /api/folders/ HTTP/1.1" 200 2

[19/May/2025 02:35:31] "GET /api/sidebar/ HTTP/1.1" 200 61



The image shows a 'Create New Note' form. It has a dark background. The form contains the following elements:

- Title ***: A text input field containing the text 'Тест'.
- Content ***: A large text area containing the text 'Тест'.
- Folder**: A dropdown menu with the text 'Select a folder for this note'.
- Tags**: A text input field.
- Buttons**: 'Cancel' and 'Create' buttons at the bottom.

Рисунок 5.3 – Окно создаваемой заметки

При создании заметки сервер получает запрос на запись: "POST /api/notes/ HTTP/1.1" 201 159. Заметка успешно создается и появляется в базе данных (рисунок 5.4).

Action:

▼

Go

0 of 3 selected

☐	TITLE	USER	FOLDER	CREATED AT	UPDATED AT
☐	Tect	Tect	-	May 18, 2025, 11:42 p.m.	May 18, 2025, 11:51 p.m.

Рисунок 5.4 – Запись новой заметки в базе данных

При изменении заголовка заметки получаем запрос: 19/May/2025 02:56:15] "PUT /api/notes/11/ HTTP/1.1" 200 180. Проверим базу данных (рисунок 5.5).

Action:

▼

Go

0 of 3 selected

☐	TITLE	USER	FOLDER	CREATED AT	UPDATED AT
☐	Тест измененный	Тест	-	May 18, 2025, 11:42 p.m.	May 18, 2025, 11:56

Рисунок 5.5 – Измененная запись о заметке

При удалении заметки получим следующие два запроса к серверу:

[19/May/2025 03:01:24] "DELETE /api/notes/11/ HTTP/1.1" 204 0

[19/May/2025 03:01:24] "GET /api/notes/ HTTP/1.1" 200 2

Первый запрос на удаление заметки, а второй на получение обновленного списка заметок. В базе данных остались только старые заметки от другого пользователя, что показано на рисунке 5.6.

Action:

Go

0 of 2 selected

☐	TITLE	USER	FOLDER	CREATED AT	UPDATED AT
☐	Тестовая заметка	admin	Подпапка	April 23, 2025, 9:51 p.m.	April 24, 2025, 10:25 p.m.
☐	Еще заметка	admin	Тестовая директория	April 23, 2025, 11:19 p.m.	April 24, 2025, 10:23 p.m.

2 notes

Рисунок 5.6 – Список заметок в базе данных

5.3 Тестирование тегов

При открытии вкладки “Tags” отправляется запрос на получение списка тегов:

[19/May/2025 03:09:07] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 03:09:07] "GET /api/structure/ HTTP/1.1" 200 148

[19/May/2025 03:09:07] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 03:09:07] "GET /api/sidebar/ HTTP/1.1" 200 206

При создании новой заметки поступают запросы:

[19/May/2025 03:12:04] "POST /api/tags/ HTTP/1.1" 201 49

[19/May/2025 03:12:04] "GET /api/tags/ HTTP/1.1" 200 51

Первый запрос к серверу на создание нового тега, а второй на получение нового списка тегов от базы данных. Проверка базы данных с новым тегом показана на рисунке 5.7.

Action:	-----	Go	0 of 2 selected
☐	NAME	▲	USER
☐	Тестовая заметка		Тест
☐	тест		admin
2 tags			

Рисунок 5.7 – Таблица базы данных с тегами

При попытке создать уже существующий тег, что показано на рисунке 5.8 получим следующие запрос и ответ: [19/May/2025 03:13:13] "POST /api/tags/ HTTP/1.1" 500 206851. Ошибка 500 означает ошибку сервера, в том числе и базы данных.

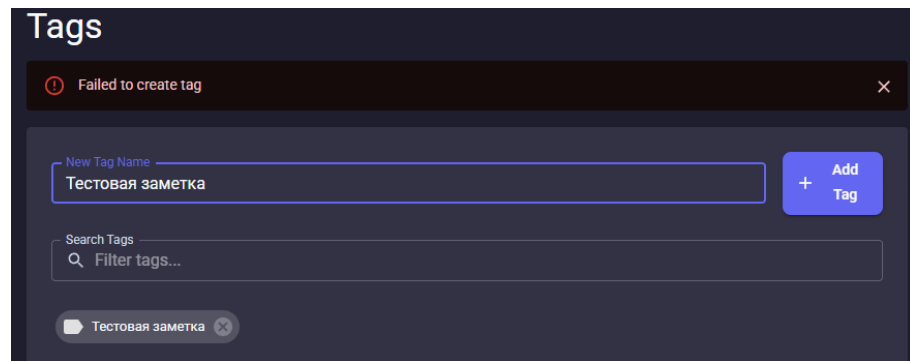


Рисунок 5.8 – Ошибка создания тега

При открытии тега происходит поиск всех заметок, в которых имеется данный тег, и сервер получает следующий запрос: [19/May/2025 03:22:53] "GET /api/notes/?tags=%D0%A2%D0%B5%D1%81%D1%82%D0%BE%D0%B2%D0%B0%D1%8F+%D0%B7%D0%B0%D0%BC%D0%B5%D1%82%D0%BA%D0%B0 HTTP/1.1" 200 210.

Для проверки функции была создана заметка с тегом. По итогу имеем результат, показанный на рисунке 5.9.

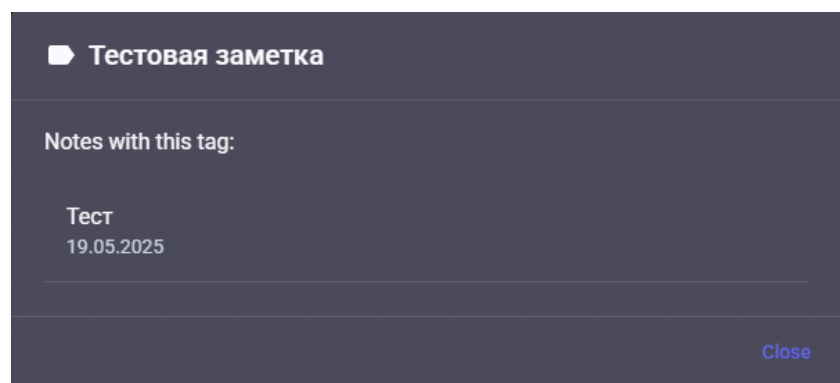


Рисунок 5.9 – Список заметок, имеющих подходящий тег

При удалении тега получаем следующие запрос и ответы от сервера:

[19/May/2025 03:29:59] "DELETE /api/tags/5/ HTTP/1.1" 204 0

[19/May/2025 03:29:59] "GET /api/tags/ HTTP/1.1" 200 2

Первый запрос к серверу на удаление существующего тега, а второй на получение нового списка тегов. Проверка базы данных показана на рисунке 5.10.

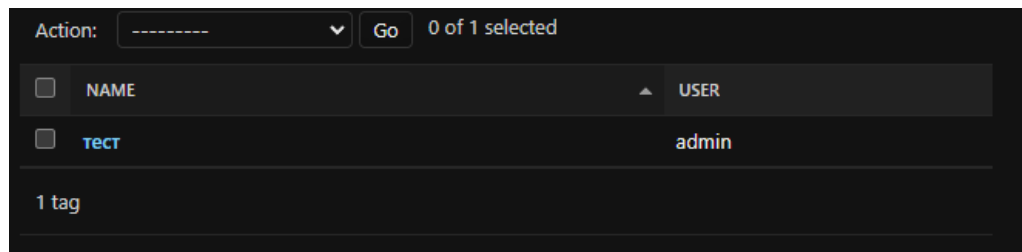


Рисунок 5.10 – Обновленный список тегов в базе данных

5.4 Тестирование папок

При открытии вкладки “Folders” отправляется запрос на получение списка существующих папок:

```
[19/May/2025 03:36:48] "GET /api/folders/ HTTP/1.1" 200 296
[19/May/2025 03:36:48] "GET /api/structure/ HTTP/1.1" 200 148
[19/May/2025 03:36:48] "GET /api/folders/ HTTP/1.1" 200 296
[19/May/2025 03:36:49] "GET /api/sidebar/ HTTP/1.1" 200 365
```

При создании новой заметки поступают запросы:

```
[19/May/2025 11:53:30] "POST /api/folders/ HTTP/1.1" 201 136
[19/May/2025 11:53:30] "GET /api/tags/ HTTP/1.1" 200 2
[19/May/2025 11:53:30] "GET /api/structure/ HTTP/1.1" 200 134
[19/May/2025 11:53:30] "GET /api/tags/ HTTP/1.1" 200 2
[19/May/2025 11:53:30] "GET /api/folders/6/ HTTP/1.1" 200 143
[19/May/2025 11:53:30] "GET /api/sidebar/ HTTP/1.1" 200 363
[19/May/2025 11:53:30] "GET /api/folders/6/ HTTP/1.1" 200 143
[19/May/2025 11:53:30] "GET /api/notes/?folder=6 HTTP/1.1" 200 2
[19/May/2025 11:53:30] "GET /api/notes/?folder=6 HTTP/1.1" 200 2
```

Первый запрос к серверу на создание новой папки, остальные запросы отправлены для получения обновленной информации для боковой панели, а также запрос на получение заметок, относящихся к папке. Проверка базы данных с новой папкой показана на рисунке 5.11.

Action:

Go

0 of 4 selected

☐	NAME	USER	PARENT	CREATED AT	UPDATED AT
☐	Новая папка	Тест	Тестова папка	May 19, 2025, 8:53 a.m.	May 19, 2025, 8:53 a.m.
☐	Подпапка	admin	Тестовая директория	April 24, 2025, 10:24 p.m.	April 24, 2025, 10:24 p.m.
☐	Тестова папка	Тест	-	May 19, 2025, 8:52 a.m.	May 19, 2025, 8:52 a.m.
☐	Тестовая директория	admin	-	April 24, 2025, 10:10 p.m.	April 24, 2025, 10:10 p.m.

Рисунок 5.11 – Таблица базы данных с папками

При открытии папки происходят следующие запросы:

[19/May/2025 12:36:59] "GET /api/tags/ HTTP/1.1" 200 2

[19/May/2025 12:36:59] "GET /api/structure/ HTTP/1.1" 200 134

[19/May/2025 12:36:59] "GET /api/folders/6/ HTTP/1.1" 200 143

[19/May/2025 12:36:59] "GET /api/sidebar/ HTTP/1.1" 200 363

[19/May/2025 12:36:59] "GET /api/notes/?folder=6 HTTP/1.1" 200 2

Все запросы направлены на получение информации о папке и хранящихся в ней заметок. Для проверки одна из заметок была добавлена в папку, результат запроса показан на рисунке 5.12.

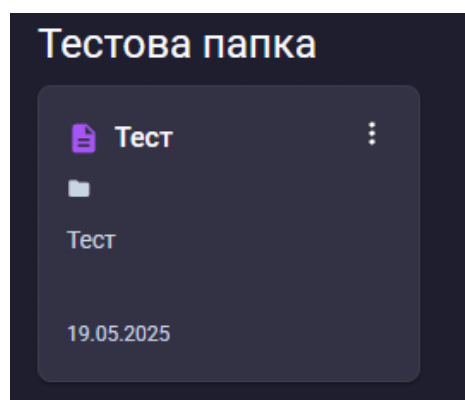


Рисунок 5.12 – Заметка добавленная в папку

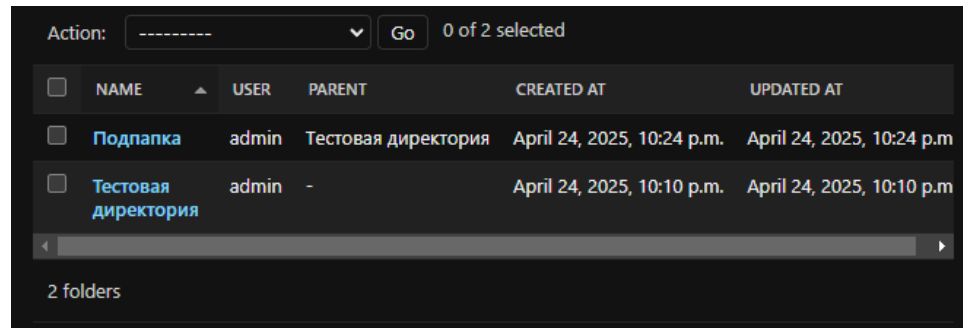
При удалении папки получаем следующие запрос и ответы от сервера:

[19/May/2025 13:04:51] "DELETE /api/folders/7/ HTTP/1.1" 204 0

[19/May/2025 13:04:51] "GET /api/structure/ HTTP/1.1" 200 226

[19/May/2025 13:04:51] "GET /api/sidebar/ HTTP/1.1" 200 360

Первый запрос к серверу на удаление существующей папки, а второй и третий на получение обновленного списка папок в боковой панели. Проверка базы данных показана на рисунке 5.13.



☐	NAME	USER	PARENT	CREATED AT	UPDATED AT
☐	Подпапка	admin	Тестовая директория	April 24, 2025, 10:24 p.m.	April 24, 2025, 10:24 p.m.
☐	Тестовая директория	admin	-	April 24, 2025, 10:10 p.m.	April 24, 2025, 10:10 p.m.

2 folders

Рисунок 5.13 – Обновленный список папок в базе данных

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана серверная часть веб-приложения для управления заметками, ориентированного на образовательные цели. Проведённый системный анализ позволил определить функциональные требования, выделить подсистемы и установить их взаимодействие с внешней средой. Выбор фреймворка Django и языка Python обоснован их преимуществами в быстрой разработке, безопасности и масштабируемости, что подтверждено сравнением с альтернативными технологиями.

Реализованное приложение включает REST API с поддержкой JWT-аутентификации, CRUD-операций для заметок, папок и тегов, а также иерархической организации данных. Тестирование подтвердило работоспособность всех подсистем, корректность обработки запросов и высокий уровень безопасности. Использование SQLite в качестве СУБД обеспечило простоту разработки, с возможностью перехода на PostgreSQL для масштабирования.

Разработанная система соответствует требованиям образовательного процесса, предоставляя удобный инструмент для управления заметками. Модульная архитектура и принципы системотехники, такие как единство, декомпозиция и модульность, обеспечивают гибкость для дальнейшего расширения функциональности, например, добавления совместного редактирования или интеграции с внешними сервисами. Результаты работы могут быть использованы в образовательных учреждениях, а также служить основой для дальнейших исследований в области веб-разработки.

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API	Application Programming Interface (программный интерфейс приложения)
CRUD	Create, Read, Update, Delete (создание, чтение, обновление, удаление)
CSRF	Cross-Site Request Forgery (межсайтовая подделка запросов)
CORS	Cross-Origin Resource Sharing (междоменный обмен ресурсами)
DRY	Don't Repeat Yourself (не повторяй себя).
HTTP	HyperText Transfer Protocol (протокол передачи гипертекста).
JSON	JavaScript Object Notation (формат обмена данными).
JWT	JSON Web Token (токен для аутентификации).
MVT	Model-View-Template (архитектурный паттерн Django).
ORM	Object-Relational Mapping (объектно-реляционное отображение).
REST	Representational State Transfer (архитектурный стиль API).
SPA	Single Page Application (одностраничное приложение).
SQL	Structured Query Language (язык структурированных запросов).
СУБД	Система управления базами данных.
XSS	Cross-Site Scripting (межсайтовый скриптинг).

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Заяц, А. М. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учеб. пособие для вузов / А. М. Заяц, Н. П. Васильев. – 3-е изд., стер. – Санкт-Петербург : Лань, 2021. – 120 с
2. Зубкова, Т. М. Технология разработки программного обеспечения / Т. М. Зубкова. – 3-е изд., стер. – Санкт-Петербург : Лань, 2023. – 252 с.
3. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – СПб. : Питер, 2018. – 464 с.
4. Бизли, Д. Python. Исчерпывающее руководство: учеб. пособие / Д. Бизли. – СПб: Питер, 2023. – 368 с.
5. Документация Django. Начало работы с Django: [Электронный ресурс]. URL: <https://www.djangoproject.com/start/>. (Дата обращения: 21.04.2025).
6. Лутц, М. Изучаем Python / М. Лутц – 5-е изд., стер. – СПб. : Диалектика, 2019. – 832 с.
7. Гринберг, М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг ; пер с англ. А.Н. Киселев. – М. : ДМК Пресс, 2016. – 272 с.
8. Ньюман, С. Создание микросервисов / С. Ньюман ; пер. с англ. С. Черников – 2-е изд., доп. – СПб. : Питер, 2023. – 624 с.
9. SaaS объяснено [Электронный ресурс]. URL: <https://www.techtarget.com/whatis/definition/SaaS-software-as-a-service>. (Дата обращения: 25.04.2025).
10. Фаулер, М. Шаблоны архитектуры корпоративных приложений / М. Фаулер. – испр. – СПб. : Диалектика, 2019. – 546 с.
11. Антао, Т. Сверхбыстрый Python : руководство / Т. Антао ; пер. с англ. А. Ю. Гинько. – М. : ДМК Пресс, 2023. – 370 с.

12. Бек, К. Экстремальное программирование. Разработка через тестирование / К. Бек ; пер. с англ. П. Анджан. – СПб. : Питер, 2017. – 251 с.
13. Коун, М. Применение пользовательских историй: гибкая разработка программного обеспечения / М. Коун. – СПб. : Диалектика, 2019. – 258 с.
14. Хамбл, Дж. Непрерывное развертывание ПО: автоматизация процессов сборки, тестирования и внедрения новых версий программ / Дж. Хамбл, Д. Фарли ; пер. с англ. А.Г. Сысолюк – СПб. : Диалектика, 2017. – 434 с.
15. Пихлер, Р. Управление продуктом в Scrum. Agile-методы для вашего бизнеса / Р. Пихлер ; пер. с англ. А. Коробейников. – М. : МИФ, 2010. – 160 с.
16. Бизли, Д. Python. Книга рецептов / Д. Бизли, Б.К. Джонс ; пер. с англ. Б.В. Уваров. – М. : ДМК Пресс, 2019. – 650с.
17. Шоу, З. Легкий способ выучить Python 3 еще глубже / З. Шоу ; пер. с англ. М Райтман. – М. : Эксмо, 2018. – 273 с.
18. Бас, Л. Архитектура программного обеспечения на практике / Л. Бас, П. Клементс, Р. Казман. – 2-е изд. – СПб. : Питер, 2006. – 464 с.
19. Ховард М. Уязвимости в программном коде и борьба с ними / М. Ховард, Д. Лебланк, Д. Виега ; пер. с англ. А.А. Слинкин – М. : ДМК Пресс, 2011 – 290 с.
20. Хант, А. Программист-прагматик / А. Хант, Д. Томас ; пер. с англ. И.В. Берштейн – СПб. : Диалектика, 2020. – 368 с.
21. Ошерове, Р. Искусство юнит-тестирования с примерами на JavaScript. / Р. Ошерове. – 3-е изд. – СПб. : Питер, 2025. – 320 с.
22. Фаулер, М. Рефакторинг: Улучшение проекта существующего кода / М. Фаулер; пер. с англ. И.В. Красиков. – СПб. : Диалектика, 2019. – 448 с.
23. Гуриков, С. Р. Основы алгоритмизации и программирования на Python : учебное пособие / С.Р. Гуриков. – М. : ИНФРА-М, 2025. – 343 с.
24. Вепрев, С. Б. Введение в Django : краткий учебный курс / С. Б. Вепрев, В. В. Крылов. – М. : ИНТУИТ, 2016. - 261 с.

25. Меле, А. Django 4 в примерах / А. Меле. – М. : ДМК Пресс, 2023. - 802 с.
26. Лоре, А. Проектирование веб-API : практическое руководство / А. Лоре ; пер. с англ. Д. А. Беликова. – М. : ДМК Пресс, 2020. - 440 с.
27. Жердев, А. А. Оптимизация клиент-серверных приложений : методические указания / А. А. Жердев. – М. : Издательский Дом НИТУ «МИСиС», 2023. - 20 с.
28. Лисьев, Г. А. Программное обеспечение компьютерных сетей и web-серверов : учебное пособие / Г.А. Лисьев, П.Ю. Романов, Ю.И. Аскерко. – М. : ИНФРА-М, 2023. – 145 с.
29. Осипов, Д. Л. Технологии проектирования баз данных / Д. Л. Осипов. – М. : ДМК Пресс, 2019. – 498 с.
30. Гринченко, Н. Н. Базы данных. Проектирование моделей данных : учебник / Н. Н. Гринченко, Н. И. Хизриева, С. Н. Баранова. – Рязань : РГРТУ, 2024. – 260 с.
31. Парамонов, А. И. Алгоритмизация и компьютерные вычисления на языках программирования высокого уровня : учебно-методическое пособие / А. И. Парамонов, А. Г. Савенко. – БГУИР : БГУИР, 2024. – 212 с.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ ПРОГРАММЫ

```

from django.contrib import admin
from .models import Tag, Folder, Note
@admin.register(Tag)
class TagAdmin(admin.ModelAdmin):
    list_display = ('name', 'user')
    list_filter = ('user',)
    search_fields = ('name',)
@admin.register(Folder)
class FolderAdmin(admin.ModelAdmin):
    list_display = ('name', 'user', 'parent', 'created_at', 'updated_at')
    list_filter = ('user', 'created_at', 'updated_at')
    search_fields = ('name',)
    raw_id_fields = ('parent',)
@admin.register(Note)
class NoteAdmin(admin.ModelAdmin):
    list_display = ('title', 'user', 'folder', 'created_at', 'updated_at')
    list_filter = ('user', 'folder', 'created_at', 'updated_at', 'tags')
    search_fields = ('title', 'content')
    raw_id_fields = ('folder',)
    filter_horizontal = ('tags',)
from django.apps import AppConfig
/notes_api/apps.py
from django.apps import AppConfig
class NotesApiConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'notes_api'
/notes_api/filters.py
import django_filters
from django.db.models import Q
from .models import Note, Tag
class NoteFilter(django_filters.FilterSet):
    search = django_filters.CharFilter(method='filter_search')
    tags = django_filters.CharFilter(method='filter_tags')
    created_after = django_filters.DateTimeFilter(field_name='created_at', lookup_expr='gte')
    created_before = django_filters.DateTimeFilter(field_name='created_at', lookup_expr='lte')

```

```

class Meta:
    model = Note
    fields = ['folder', 'search', 'tags', 'created_after', 'created_before']
    def filter_search(self, queryset, name, value):
        if not value:
            return queryset
        return queryset.filter(
            Q(title__icontains=value) | Q(content__icontains=value)
        )

    def filter_tags(self, queryset, name, value):
        if not value:
            return queryset
        # Split comma-separated tag names
        tag_names = [tag.strip() for tag in value.split(',')]
        # Filter notes that have all the specified tags
        return queryset.filter(tags__name__in=tag_names).distinct()

/notes_api/models.py
from django.db import models
from django.contrib.auth.models import User
# Create your models here.
class Tag(models.Model):
    name = models.CharField(max_length=50)
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='tags')
    class Meta:
        unique_together = ['name', 'user']
        ordering = ['name']
    def __str__(self):
        return self.name

class Folder(models.Model):
    name = models.CharField(max_length=100)
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='folders')
    parent = models.ForeignKey('self', on_delete=models.CASCADE, null=True, blank=True,
related_name='children')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta:
        ordering = ['name']

```

```

def __str__(self):
    return self.name
class Note(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='notes')
    folder = models.ForeignKey(Folder, on_delete=models.SET_NULL, null=True, blank=True,
related_name='notes')
    tags = models.ManyToManyField(Tag, blank=True, related_name='notes')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta:
        ordering = ['-updated_at']
    def __str__(self):
        return self.title
/notes_api/serializers.py
from rest_framework import serializers
from .models import Tag, Folder, Note
from django.contrib.auth.models import User
class TagSerializer(serializers.ModelSerializer):
    class Meta:
        model = Tag
        fields = ['id', 'name']
        read_only_fields = ['id']
    def create(self, validated_data):
        # Associate with current user
        validated_data['user'] = self.context['request'].user
        return super().create(validated_data)
class FolderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Folder
        fields = ['id', 'name', 'parent', 'created_at', 'updated_at']
        read_only_fields = ['id', 'created_at', 'updated_at']
    def create(self, validated_data):
        # Associate with current user
        validated_data['user'] = self.context['request'].user
        return super().create(validated_data)
class NoteSerializer(serializers.ModelSerializer):

```

```

tags = TagSerializer(many=True, required=False, read_only=True)
tag_ids = serializers.PrimaryKeyRelatedField(
    many=True, write_only=True, required=False, queryset=Tag.objects.all(), source='tags'
)
tag_names = serializers.ListField(
    child=serializers.CharField(max_length=50),
    write_only=True,
    required=False
)
class Meta:
    model = Note
    fields = [
        'id', 'title', 'content', 'folder', 'tags', 'tag_ids',
        'tag_names', 'created_at', 'updated_at'
    ]
    read_only_fields = ['id', 'created_at', 'updated_at']
    def create(self, validated_data):
        # Handle tag names if provided
        tag_names = validated_data.pop('tag_names', [])
        # Associate with current user
        user = self.context['request'].user
        validated_data['user'] = user
        # Create the note
        note = super().create(validated_data)
        # Create tags if they don't exist and add to note
        if tag_names:
            for tag_name in tag_names:
                tag, created = Tag.objects.get_or_create(
                    name=tag_name,
                    user=user
                )
                note.tags.add(tag)

        return note
    def update(self, instance, validated_data):
        # Handle tag names if provided
        tag_names = validated_data.pop('tag_names', [])

```



```

# Проверяем, является ли это частичным обновлением только для поля folder
is_only_folder_update = len(validated_data) == 1 and 'folder' in validated_data
# Update the note
note = super().update(instance, validated_data)
# Create tags if they don't exist and add to note
if tag_names:
    # Если это только обновление папки, не меняем теги
    if not is_only_folder_update:
        # Сначала очистим существующие теги, если это полное обновление
        note.tags.clear()
    # Добавляем новые теги
    for tag_name in tag_names:
        tag, created = Tag.objects.get_or_create(
            name=tag_name,
            user=self.context['request'].user
        )
        note.tags.add(tag)
    return note

class FolderStructureSerializer(serializers.ModelSerializer):
    children = serializers.SerializerMethodField()
    notes = NoteSerializer(many=True, read_only=True)
    class Meta:
        model = Folder
        fields = ['id', 'name', 'children', 'notes']
    def get_children(self, obj):
        # Recursively serialize children folders
        children = Folder.objects.filter(parent=obj)
        serializer = FolderStructureSerializer(children, many=True, context=self.context)
        return serializer.data

class SidebarSerializer(serializers.ModelSerializer):
    folders = serializers.SerializerMethodField()
    recent_notes = serializers.SerializerMethodField()
    class Meta:
        model = User
        fields = ['id', 'username', 'folders', 'recent_notes']
    def get_folders(self, obj):
        # Get top-level folders
        folders = Folder.objects.filter(user=obj, parent=None)

```

```

        return FolderSerializer(folders, many=True).data
    def get_recent_notes(self, obj):
        # Get 5 most recent notes
        notes = Note.objects.filter(user=obj).order_by('-updated_at')[:5]
        return NoteSerializer(notes, many=True).data

/notes_api/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import (
    TagViewSet, FolderViewSet, NoteViewSet,
    FolderStructureView, SidebarView
)
router = DefaultRouter()
router.register(r'tags', TagViewSet, basename='tag')
router.register(r'folders', FolderViewSet, basename='folder')
router.register(r'notes', NoteViewSet, basename='note')
urlpatterns = [
    path("", include(router.urls)),
    path('structure/', FolderStructureView.as_view(), name='folder-structure'),
    path('sidebar/', SidebarView.as_view(), name='sidebar'),
]

/notes_api/views.py
from django.shortcuts import render
from rest_framework import viewsets, generics, status
from rest_framework.response import Response
from rest_framework.decorators import action
from rest_framework.permissions import IsAuthenticated
from django.shortcuts import get_object_or_404
from django.db.models import RestrictedError
from .models import Tag, Folder, Note
from .serializers import (
    TagSerializer, FolderSerializer, NoteSerializer,
    FolderStructureSerializer, SidebarSerializer
)
from .filters import NoteFilter
class TagViewSet(viewsets.ModelViewSet):
    serializer_class = TagSerializer
    permission_classes = [IsAuthenticated]

```

```

def get_queryset(self):
    return Tag.objects.filter(user=self.request.user)

class FolderViewSet(viewsets.ModelViewSet):
    serializer_class = FolderSerializer
    permission_classes = [IsAuthenticated]
    def get_queryset(self):
        queryset = Folder.objects.filter(user=self.request.user)
        parent = self.request.query_params.get('parent')
        if parent:
            if parent == 'null':
                queryset = queryset.filter(parent=None)
            else:
                queryset = queryset.filter(parent=parent)
        return queryset
    def destroy(self, request, *args, **kwargs):
        folder = self.get_object()
        # Check if folder has children or notes
        if folder.children.exists() or folder.notes.exists():
            return Response(
                {"detail": "Cannot delete folder that contains notes or subfolders."},
                status=status.HTTP_400_BAD_REQUEST
            )
        return super().destroy(request, *args, **kwargs)

class NoteViewSet(viewsets.ModelViewSet):
    serializer_class = NoteSerializer
    permission_classes = [IsAuthenticated]
    filterset_class = NoteFilter
    search_fields = ['title', 'content']
    ordering_fields = ['created_at', 'updated_at', 'title']
    def get_queryset(self):
        return Note.objects.filter(user=self.request.user).prefetch_related('tags').select_related('folder')
    def partial_update(self, request, *args, **kwargs):
        kwargs['partial'] = True
        instance = self.get_object()
        # Если обновляется только папка, обрабатываем специальным образом
        if len(request.data) == 1 and 'folder' in request.data:
            serializer = self.get_serializer(
                instance,

```

```

        data=request.data,
        partial=True,
        context={'request': request}
    )
    serializer.is_valid(raise_exception=True)
    self.perform_update(serializer)
    return Response(serializer.data)

    return super().partial_update(request, *args, **kwargs)

class FolderStructureView(generics.GenericAPIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        # Get all top-level folders for the user (where parent is None)
        root_folders = Folder.objects.filter(user=request.user, parent=None)
        serializer = FolderStructureSerializer(root_folders, many=True, context={'request': request})
        return Response(serializer.data)

class SidebarView(generics.GenericAPIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        serializer = SidebarSerializer(request.user, context={'request': request})
        return Response(serializer.data)

/notes_project/asgi.py
import os
from django.core.asgi import get_asgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'notes_project.settings')
application = get_asgi_application()

/notes_project/settings.py
SECRET_KEY = 'django-insecure-8mp%^5aw9twci^e#sb_=bsu2*wwnd5+z+c(4@eq&cc+6y(-i7i'
# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# CORS settings
CORS_ALLOW_ALL_ORIGINS = True
CORS_ALLOW_CREDENTIALS = True
CORS_EXPOSE_HEADERS = ['Content-Type', 'X-CSRFToken']
CORS_ALLOW_METHODS = ['DELETE', 'GET', 'OPTIONS', 'PATCH', 'POST', 'PUT']
CORS_ALLOW_HEADERS = ['accept', 'accept-encoding', 'authorization', 'content-type', 'dnt',
'origin', 'user-agent', 'x-csrf-token', 'x-requested-with']

```

```

# Application definition
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Third party apps
    'rest_framework',
    'django_filters',
    'corsheaders',
    # Local apps
    'notes_api',
    'accounts',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'corsheaders.middleware.CorsMiddleware',    # CORS middleware должен быть перед
CommonMiddleware
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'notes_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
            ]
        }
    }
]

```

```

        'django.contrib.messages.context_processors.messages',
    ],
},
],
WSGI_APPLICATION = 'notes_project.wsgi.application'
# Database
# https://docs.djangoproject.com/en/5.1/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
# Password validation
# https://docs.djangoproject.com/en/5.1/ref/settings/#auth-password-validators
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
# REST Framework settings
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],

```

```

'DEFAULT_FILTER_BACKENDS': [
    'django_filters.rest_framework.DjangoFilterBackend',
    'rest_framework.filters.SearchFilter',
    'rest_framework.filters.OrderingFilter',
],
}
# JWT settings
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(days=1),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': False,
    'BLACKLIST_AFTER_ROTATION': False,
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': SECRET_KEY,
    'VERIFYING_KEY': None,
    'AUTH_HEADER_TYPES': ('Bearer',),
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
    'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',),
    'TOKEN_TYPE_CLAIM': 'token_type',
}
# Internationalization
# https://docs.djangoproject.com/en/5.1/topics/i18n/
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.1/howto/static-files/
STATIC_URL = 'static/'
# Default primary key field type
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
/notes_project/urls.py
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('notes_api.urls')),

```

```

    path('api/auth/', include('accounts.urls')),
]
/notes_project/wsgi.py
import os
from django.core.wsgi import get_wsgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'notes_project.settings')
application = get_wsgi_application()
/accounts/apps.py
from django.apps import AppConfig
class AccountsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'accounts'
/accounts/serializers.py
from rest_framework import serializers
from django.contrib.auth.models import User
from django.contrib.auth.password_validation import validate_password
from rest_framework.validators import UniqueValidator
class RegisterSerializer(serializers.ModelSerializer):
    email = serializers.EmailField(
        required=True,
        validators=[UniqueValidator(queryset=User.objects.all())]
    )
    password = serializers.CharField(
        write_only=True, required=True, validators=[validate_password]
    )
    password2 = serializers.CharField(write_only=True, required=True)
    class Meta:
        model = User
        fields = ('username', 'password', 'password2', 'email', 'first_name', 'last_name')
        extra_kwargs = {
            'first_name': {'required': False},
            'last_name': {'required': False}
        }
    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({"password": "Password fields didn't match."})
        return attrs
    def create(self, validated_data):

```



```

user = User.objects.create(
    username=validated_data['username'],
    email=validated_data['email'],
    first_name=validated_data.get('first_name', ''),
    last_name=validated_data.get('last_name', '')
)
user.set_password(validated_data['password'])
user.save()

return user

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name', 'last_name']
        read_only_fields = ['id']

/accounts/urls.py
from django.urls import path
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)
from .views import RegisterView, UserProfileView
urlpatterns = [
    path('register/', RegisterView.as_view(), name='register'),
    path('login/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    path('profile/', UserProfileView.as_view(), name='user-profile'),
]

/accounts/views.py
from django.shortcuts import render
from rest_framework import generics, permissions
from rest_framework.response import Response
from rest_framework.decorators import api_view, permission_classes
from django.contrib.auth.models import User
from .serializers import RegisterSerializer, UserSerializer
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    permission_classes = [permissions.AllowAny]
    serializer_class = RegisterSerializer

```

```
class UserProfileView(generics.RetrieveUpdateAPIView):  
    serializer_class = UserSerializer  
    permission_classes = [permissions.IsAuthenticated]  
    def get_object(self):  
        return self.request.user
```